

Linux File System Hierarchy (FHS)

The Linux File System Hierarchy (FHS) organizes files in a tree structure starting from the root directory (/), with standard subdirectories for different functions like /bin (essential commands), /etc (config files), /home (user data), /var (variable data/logs), /usr (user programs/data), and /lib (essential libraries) to ensure consistency across distributions, with key virtual file systems like /proc and /sys for kernel information

/ (Root): The top of the hierarchy; everything starts here.

- /bin: Essential user command binaries (e.g., ls, cp).
- /sbin: Essential system administration binaries (e.g., fdisk, iptables).
- /etc: System-wide configuration files.
- /home: User home directories (e.g., /home/user1).
- /lib, /lib64: Essential shared libraries for binaries in /bin and /sbin.
- /opt: Optional, third-party application software.
- /tmp: Temporary files, often cleared on reboot.
- /var: Variable data, like logs (/var/log), mail, and print queues.
- /boot: Files needed to boot the system (kernel, GRUB).
- /dev: Device files representing hardware.
- /proc: Virtual filesystem for process and kernel information (e.g., /proc/cpuinfo).
- /sys: Virtual filesystem for kernel/hardware interface.
- /media, /mnt: Mount points for removable media (USB drives) or temporary filesystem mounts
- /root: The root user's home directory.
- /usr: Secondary hierarchy for user-related programs and data

```
$ tree -d -L 1 /
```

Absolute Path

An absolute path is the complete and unambiguous location of a file or directory, regardless of where you are in the file system

Example: If a file named report.txt is located in a user's documents folder, the absolute path would be /home/alex/documents/report.txt

Relative Path

A relative path specifies the location of a file or directory in relation to your current working directory.

Example: If your current working directory is /home/alex, and you want to access report.txt in the documents subdirectory, the relative path is documents/report.txt or ./documents/report.txt. If you were in /home/alex/downloads, the path would be ../documents/report.txt

pwd print present working directory.
cd change directory

cd ~ will put you in your home directory
cd .. To go to the parent directory (one level above)
cd - go to the previous directory.
ls list the contents of a directory(check options with **ls -help**)

All files are case sensitive

Files on Linux (or any Unix) are case sensitive. This means that FILE1 is different from file1, and /etc/hosts is different from /etc/Hosts

Everything is a file

A directory is a special kind of file, but it is still a (case sensitive!) file. Each terminal window (for example /dev/pts/4), any hard disk or partition (for example /dev/sdb1) and any process are all represented somewhere in the file system as a file.

File Command

The file utility determines the file type. Linux does not use extensions to determine the file type. The command line does not care whether a file ends in .txt or .pdf. As a system administrator, you should use the file command to determine the file type. Here are some examples on a typical Linux system.

```
$ file pic33.png
pic33.png: PNG image data, 3840 x 1200, 8-bit/color RGBA, non-interlaced
```

```
$ file /etc/passwd
/etc/passwd: ASCII text
```

```
$ file HelloWorld.c
HelloWorld.c: ASCII C program text
```

The file command uses a magic file that contains patterns to recognize file types. The magic file is located in /usr/share/file/magic. Type **man 5 magic** for more information. It is interesting to point out file -s for special files like those in /dev and /proc.

```
# file /dev/sda
/dev/sda: block special
```

```
# file -s /dev/sda
/dev/sda: x86 boot sector; partition 1: ID=0x83, active, starthead...
```

```
# file /proc/cpuinfo
/proc/cpuinfo: empty
```

```
# file -s /proc/cpuinfo
/proc/cpuinfo: ASCII C++ program text
```

Other commands

Check with options (or **man command** , **command --help**)

mkdir, **rmdir**, **touch**, **rm**, **cp**, **mv**

working with file contents

we will look at the contents of text files with **head**, **tail**, **cat**, **tac**, **more**, **less** and **strings**.

head to display the first ten lines of a file.

The head command can also display the first n lines of a file. (**head -5 /etc/passwd**)

And head can also display the first n bytes.(**head -c14 /etc/passwd**)

tail similar to head, the tail command will display the last ten lines of a file.

cat to display a file on the screen. If the file is longer than the screen, it will scroll to the end.

cat is short for concatenate. One of the basic uses of cat is to concatenate files into a bigger (or complete) file. (\$cat part1 part2 part3 >all)

You can use cat to create flat text files.

Type the cat > winter.txt command as shown in the screenshot below.

Then type one or more lines, finishing each line with the enter key. After the last line, type and hold the Control (Ctrl) key and press d. The Ctrl d key combination will send an EOF (End of File) to the running process ending the cat command.

You can choose an end marker for cat with << as is shown in this screenshot. This construction is called a here directive and will end the cat command.

```
$ cat > hot.txt <<stop
> It is hot today!
> Yes it is summer.
> stop
```

tac (cat backwards).

Strings With the strings command you can display readable ascii strings found in (binary) files.

This example locates the ls binary then displays readable strings in the binary file (output is truncated).

```
$which ls
/bin/ls
$ strings /bin/ls
/lib/ld-linux.so.2 librt.so.1
```

more and less

The more and less commands are Linux pagers used to view the contents of large text files one screen at a time, rather than loading the entire file into memory. The primary difference is that less is an enhanced version of more that allows both forward and backward navigation, while more generally only permits forward movement.

```
$more [options] filename
or
$command_output | more
```

The **less** command is a more advanced pager and the generally preferred utility today because it offers more features and flexibility (a common joke in the Linux community is "less is more"). It starts faster with large files as it doesn't read the entire file into memory first.

```
$less [options] filename
or
$command_output | less
```

Examples & Navigation:

View a file with line numbers: less -N filename.txt
displays line numbers alongside the file content.

Open a file and jump to the first occurrence of a pattern:

```
less -p "pattern" filename.txt
starts the display at the first match for "pattern".
```

Pipe command output and exit automatically if the output fits on one screen:
dmesg | less -F

Navigation keys while in less:

- Spacebar or PageDown: Move to the next page.
- b or PageUp: Move to the previous page.

- Down Arrow or j key: Move down one line.
- Up Arrow or k key: Move up one line.
- / followed by a pattern: Search forward for a specific text string. Press n to go to the next match or N to go to the previous match.
- q key: Quit and return to the terminal prompt

single quotes

You can prevent the removal of white spaces by quoting the spaces. The contents of the quoted string are considered as one argument. In the screenshot below the echo receives only one argument.

```
$ echo 'A line with single quotes'
A line with single quotes
```

double quotes

You can also prevent the removal of white spaces by double quoting the spaces. Same as above, echo only receives one argument.

```
$ echo "A line with double quotes"
A line with double quotes
```

external or builtin commands ?

Not all commands are external to the shell, some are builtin. External commands are programs that have their own binary and reside somewhere in the file system. Many external commands are located in /bin or /sbin. Builtin commands are an integral part of the shell program itself.

type

To find out whether a command given to the shell will be executed as an external command or as a builtin command, use the type command.

```
$ type cd
cd is a shell builtin
```

```
$ type cat
cat is /bin/cat
```

As you can see, the cd command is builtin and the cat command is external. You can also use this command to show you whether the command is aliased or not.

```
$ type ls
ls is aliased to `ls -color=auto'
```

which

The which command will search for binaries in the \$PATH environment variable (variables will be explained later). In the screenshot below, it is determined that cd is builtin, and ls, cp, rm, mv, mkdir, pwd, and which are external commands.

```
$ which cp ls cd mkdir pwd
```

aliases

create an alias

The shell allows you to create aliases. Aliases are often used to create an easier to remember name for an existing command or to easily supply parameters.

```
$ cat count.txt
one
two
three
$ alias dog=tac
$ dog count.txt
three
```

```
two
one
```

unalias

You can undo an alias with the unalias command.

displaying shell expansion

You can display shell expansion with set -x, and stop displaying it with set +x. You might want to use this further on in this course, or when in doubt about exactly what the shell is doing with your command.

```
tux@tuxhost:~$ echo $USER
tux
tux@tuxhost:~$ set -x
tux@tuxhost:~$ echo $USER
+ echo tux
tux
tux@tuxhost:~$ set +x
tux@tuxhost:~$ echo $USER
tux
tux@tuxhost:~$
```

echo command

The Global Syntax of echo command with options :

```
echo [OPTION] [input string]
```

The following tabular gives you the possible options in the echo command:

Examples:

```
# echo -e "LinuxTeck \tA \tComplete \tLearning \tBlog"
```

Output: LinuxTeck A Complete Learning Blog

How to print all the files and folders?

```
# echo *
```

How to print some particular extensions of files?

```
#echo *.log
```

control operators

1. Semicolon (;)

The semicolon is used to separate multiple commands that will be executed sequentially, regardless of whether the previous command succeeded or failed.

Example:

```
echo "Hello"; echo "World"
```

This will output:

```
Hello
```

```
World
```

Even if one of the commands fails, the next one will still be executed.

2. AND (&&)

The && operator allows you to execute the second command only if the first command succeeds (returns a 0 exit status).

Example: mkdir new_folder && cd new_folder

This will create a directory called new_folder and only move into it if the directory creation was successful.

3. OR (||)

The || operator is the opposite of &&. It will execute the second command only if the first command fails (non-zero exit status).

Example: mkdir new_folder || echo "Failed to create folder"

If mkdir new_folder fails (for example, if the folder already exists), it will print Failed to create folder.

4. Pipe (|)

A pipe is used to send the output of one command as input to another command. It's one of the most commonly used operators for chaining commands.

Example: `cat file.txt | grep "hello"`

This will send the contents of `file.txt` to `grep`, which will search for the word "hello".

5. Double Ampersand (&&) with Commands

The `&&` operator is also commonly used in combination with commands to build more complex conditional logic.

Example: `make && sudo make install`

This will first run `make`. If it is successful, it will then run `sudo make install`

6. Double Pipe (||) with Commands

Similarly to `&&`, `||` can be used to chain commands based on failure. If the first command fails, the second command will execute.

Example: `echo "File does not exist" || echo "This is always executed"`

7. Grouping Commands with Parentheses (())

Parentheses are used to group commands, which are executed in a subshell. Changes in the environment (like setting variables) made inside the parentheses do not affect the current shell.

Example: `(echo "First"; echo "Second")`

This will execute both `echo` commands in a subshell.

8. Background Execution (&)

The ampersand `&` allows you to run a command in the background, so that you can continue using the terminal for other tasks while the command executes.

Example: `long_running_command &`

This runs `long_running_command` in the background.

A subshell in Linux (and Unix-like operating systems) refers to a child process created by the shell to execute a set of commands separately from the parent shell. The subshell inherits the environment of the parent shell but operates independently, which means it has its own memory space and process ID.

How to Create a Subshell in Linux

The simplest way to create a subshell is to use parentheses `()` to group commands together. When the shell executes commands inside parentheses, it runs them in a subshell.

Example: Grouping Commands in a Subshell

```
# Parent shell's working directory is /home/user
```

```
echo "Parent shell's directory: $(pwd)"
```

Output: Parent shell's directory: /home/user

Create a subshell

```
(echo "This is inside the subshell"; cd /tmp; echo "Subshell's directory: $(pwd)")
```

Output:

```
This is inside the subshell
```

```
Subshell's directory: /tmp
```

```
# Parent shell's working directory is unchanged
```

```
echo "Parent shell's directory after subshell: $(pwd)"
```

Output:

```
Parent shell's directory after subshell: /home/user
```

Explanation:

The command `cd /tmp` inside the subshell changes the working directory only within that subshell.

After the subshell terminates, the parent shell's current working directory remains unchanged.

When to Use a Subshell**1. Isolating Changes to the Environment:**

If you want to temporarily change the environment (like changing the working directory, setting variables, or modifying shell settings) and you don't want those changes to affect the parent shell, you can use a subshell.

2. Running Commands in Parallel:

Sometimes you may want to run commands in parallel without affecting the parent shell. You can execute commands in the background or in a subshell.

Run commands in the background (each in its own subshell)
(`cmd1 &`; `cmd2 &`; `cmd3 &`)

3. Command Grouping:

When you want to group multiple commands and treat them as a single unit, subshells allow you to execute a sequence of commands without affecting the current shell session.

Using a subshell to group commands together
(`result=$(echo "Hello, world!" && echo "This is a subshell")`)
`echo $result` # Output will be: Hello, world! This is a subshell

4. Redirecting Output of Multiple Commands Together:

If you want to redirect the output of multiple commands to a file or pipe, you can group them inside a subshell.

Redirect multiple commands' output to a file
(`echo "First command"; echo "Second command"`) > `output.txt`
This executes both `echo` commands in the subshell, and redirects their combined output to `output.txt`

Regular Expressions

Key Concepts of POSIX Basic Regular Expressions (BRE)

In Basic Regular Expressions (BRE), some special characters and symbols are interpreted differently from how they are in Extended Regular Expressions (ERE). The main challenge with BRE is that certain metacharacters (like `+`, `?`, `|`, etc.) are not treated as special unless they are preceded by a backslash (`\`).

Basic Metacharacters in BRE

Here are the most commonly used special characters in Basic Regular Expressions (BRE) and their meanings:

- `.` (dot) - Matches any single character except newline (`\n`).
 - Example: `a.b` matches `acb`, `axb`, but not `ab`.
- `^` (caret) - Anchors the match at the beginning of a line.
 - Example: `^abc` matches the string `abc` only if it occurs at the start of a line.
- `$` (dollar sign) - Anchors the match at the end of a line.
 - Example: `abc$` matches the string `abc` only if it occurs at the end of a line.
- `*` (asterisk) - Matches zero or more occurrences of the preceding character or group.
 - Example: `a*b` matches `b`, `ab`, `aab`, `aaab`, etc.
- `\` (backslash) - Used to escape special characters to match them literally or to introduce special sequences like character classes.
 - Example: `\.` matches a literal dot (`.`), `\\` matches a literal backslash (`\`).

- `[]` (square brackets) - Matches any one character from the enclosed set of characters.
 - Example: `[aeiou]` matches any vowel (a, e, i, o, or u).
 - You can specify ranges inside the brackets:
 - `[a-z]` matches any lowercase letter.
 - `[0-9]` matches any digit.
- `[^]` (caret inside square brackets) - Matches any one character that is NOT in the enclosed set.
 - Example: `[^0-9]` matches any character that is not a digit.
- `\{m,n\}` (braces with numbers) - Matches between m and n occurrences of the preceding element (this is specific to BRE but requires escaping the { and }).
 - Example: `a\{2,4\}` matches aa, aaa, or aaaa.

Literal Characters

In BRE, characters that are not metacharacters are matched literally. For example, a, b, and 1 would match the characters "a", "b", and "1" exactly.

Grouping in BRE

In BRE, grouping (used in ERE for logical OR and more complex patterns) requires the use of backslashes:

- `\( ... \)` - Used to group parts of a pattern.
 - Example: `\(ab\)*` matches any number of repetitions of the string ab.

Example: Using Basic Regular Expressions

- Matching a simple string:
- `echo "hello" | grep "hello"`

This matches the exact string "hello".

- Using `^` and `$` to match the beginning or end of a string:

`echo "hello" | grep "^h"`

This matches the string "hello" only if it starts with h.

- `echo "hello" | grep "o$"`

This matches the string "hello" only if it ends with o.

- Using `*` for zero or more occurrences:
- `echo "aaaab" | grep "a*b"`

This matches aaaab, as `a*` matches zero or more occurrences of a.

- Using `[]` for character classes:
- `echo "apple" | grep "[aeiou]"`

This matches apple because it contains at least one vowel (a).

- Using `[^]` for negating a character class:
- `echo "apple" | grep "[^aeiou]"`

This matches apple because it contains characters that are not vowels.

- Grouping and repetition with `\{m,n\}`:
- `echo "aaaaab" | grep "a\{2,4\}b"`

This matches aaaab because it has between 2 and 4 as before the b.

Examples of More Complex Patterns

- Matching a phone number with specific format (e.g., 123-456-7890):
- `echo "123-456-7890" | grep "^[0-9]\{3\}-[0-9]\{3\}-[0-9]\{4\}$"`

This pattern matches a phone number in the format of exactly three digits, followed by a dash, followed by three digits, another dash, and four digits.

- Matching multiple lines with `^` and `$` for beginning and end of lines (using `grep -P` for multi-line patterns):

- `grep "^foo" filename.txt` # Matches lines starting with 'foo'
- Matching a specific number of occurrences (using `\{m,n\}`):
- `echo "aaabbb" | grep "a\{2,3\}b\{2,3\}"`

This matches aaabbbb because there are between 2 and 3 as and between 2 and 3 bs.

Summary of POSIX BRE Syntax

- Metacharacters: ., *, ^, \$, [], \, and \{m,n\}.
- Literals: Any character that is not a special symbol is treated literally.
- Escaping Metacharacters: To match special characters literally, use

backslashes (\).

- Grouping: Group parts of expressions using \(... \).
- Repetition: Use * for zero or more repetitions; \{m,n\} for a specific range of repetitions.

Note: Many utilities (such as grep, sed, awk) use regular expressions in their operations. For extended regular expressions (ERE), there are more advanced features available, but these features require a different syntax (e.g., +, ?, |, etc.).

#####

SHELL

\$PS1

The \$PS1 variable determines your shell prompt

\$PATH

The \$PATH variable is determines where the shell is looking for commands to execute (unless the command is builtin or aliased). This variable contains a list of directories, separated by colons.

\$env

The env command without options will display a list of exported variables.

\$export

export is used to set or export environment variables. When you export a variable, it becomes available to any subprocess or child process you start from the current shell.

- Setting and exporting a variable:

```
export VAR="some value"
```

After this, VAR will be available to other processes started from the shell. For example:

- echo \$VAR # Will output 'some value'
- Making a variable accessible to child processes:

When you export a variable, any script or program run from the shell will inherit the variable.

\$set

The set command is used to set or display values of shell variables, functions, or both. It's more general than export, and it's used to set both environment variables (variables available to child processes) and shell variables (which are only available in the current shell session).

- Without any arguments: It shows all environment and shell variables, including functions:

- set
- Set a new variable:

set VAR=value

However, note that setting a variable this way does not make it available to child processes (unlike export).

\$ (Dollar Sign)

The \$ symbol is used in the shell to represent the value of a variable.

- Accessing variable values:

```
VAR="Hello, world!"
```

```
echo $VAR
```

This will print:

- Hello, world!

- Accessing the value of environment variables:

echo \$HOME

This will print the current user's home directory.

\$\$

\$\$ is a special variable that contains the process ID (PID) of the currently running shell. Every process in Linux has a unique PID, and \$\$ allows you to refer to the PID of the shell process that is executing the command.

Example

echo \$\$ # This will display the PID of the current shell process

HISTORY

1.View Command History:

Running the history command without any arguments will display a list of previously executed commands.

\$history

Example output:

```
1 ls
2 cd Documents
3 cat myfile.txt
4 history
```

The output shows the history number and the command that was executed.

2.Configuring History

You can control how the shell manages command history using environment variables. The two key variables are:

- HISTSIZE: The number of commands to remember in the current session. It defines how many commands are kept in memory.

- Example: export HISTSIZE=1000 keeps the last 1000 commands in memory.

- HISTFILESIZE: The number of commands to keep in the history file (usually ~/.bash_history). It defines how many commands are stored on disk.

- Example: export HISTFILESIZE=2000 will keep the last 2000 commands in the history file.

3. History File (~/.bash_history)

By default, command history is stored in a file called ~/.bash_history (for Bash shell). Each time you exit the terminal session, the commands are saved to this file. When you start a new session, the commands from this file are loaded into the history list.

- Viewing the history file:

You can view the history file directly:

- cat ~/.bash_history

4. Common history Command Options

The history command has several useful options that you can use to manipulate and interact with your command history.

1. View Specific Number of Commands

- To view the last N commands in your history (e.g., last 20 commands):
- history 20

2. Clear History

- To clear the current history from the shell session (but not from the history file):

- history -c

This will clear the history list from memory but won't modify the ~/.bash_history file unless you use additional commands (e.g., history -w).

3. Remove a Specific Command from History

- If you want to remove a specific command from history, you can use the `-d` option followed by the history number:

- `history -d 5`

This will delete the command with history number 5.

4. Append History to the History File

- To force the shell to write the current session's history to the `~/.bash_history` file immediately, you can use the `-a` option:

- `history -a`

This is useful if you've had a long session and want to ensure your history is saved before exiting.

5. Write History to the History File

- To write the history list from memory into the history file:
- `history -w`

This saves the current history list (in memory) to the history file (`~/.bash_history`), overwriting the contents of the file.

6. Read History from the History File

- To load the history from the history file into memory:
- `history -r`

This is useful if you want to reload history from the file without starting a new session.

7. Time Stamps in History

- If you want to see the timestamps for each command in history (i.e., when each command was run), you can enable this by setting the `HISTTIMEFORMAT` environment variable.

Example:

```
export HISTTIMEFORMAT="%F %T "
```

This will display the date and time alongside the history command. For example:

- `2023-01-13 14:35:12 ls`

```
2023-01-13 14:36:03 cd Documents
```

You can add this to your `~/.bashrc` file to make the change permanent.

5. Using ! to Repeat Commands

The history command allows you to execute commands from your history using the `!` (exclamation mark) followed by a number or part of a command.

1. Run a Command by History Number

You can re-execute a command by referencing its history number.

- For example:
- `!5`

This will execute the command with history number 5.

2. Run the Last Command

You can use `!!` to repeat the last command you executed.

- For example:
- `!!`

This repeats the last command you ran. It's great for when you make a small mistake and just want to quickly run the previous command again.

3. Run the Last Command Starting with a String

You can use `!string` to repeat the last command that starts with a certain string.

- For example:
- `!ls`

This will execute the last command starting with `ls`, such as `ls -l` or `ls /home`.

4. Run a Command from History by Pattern Matching

You can use `!` followed by part of the command to run a command that contains that string.

- For example:
- `!cat`

This will repeat the most recent command that contains `cat` in it.

5. Run a Command by History Number with Substring Matching

To use history numbers dynamically, you can use the `!` operator with `!number` in combination with the `!string` pattern matching.

Example:

```
history | grep "grep"
```

Then you can repeat the command: `!grep`

6. Advanced History Features

1. History Expansion (with HISTCONTROL)

You can control how duplicate commands are handled using the HISTCONTROL variable. Common options include:

- ignoredups: Ignores duplicate commands (if the same command is run consecutively, it's not saved).
- ignorespace: Ignores commands that start with a space.
- erasedups: Deletes all previous occurrences of the same command in the history file when a new command is entered.

To enable HISTCONTROL with both ignoredups and ignorespace:

```
export HISTCONTROL=ignoredups:ignorespace
```

2. History File Customization

You can customize your history behavior by modifying the following environment variables:

- HISTFILE: The file where history is saved (~/.bash_history by default).
- HISTSIZE: The number of commands stored in the history list for the current session.
- HISTFILESIZE: The number of commands saved in the history file.
- HISTIGNORE: A colon-separated list of patterns to exclude from the history.

Example to exclude ls and cd commands from history:

```
export HISTIGNORE="ls:cd"
```

3. history and .bashrc

If you want to change or fine-tune your history settings permanently, you can modify your ~/.bashrc file. For example:

```
# Ignore duplicate commands
export HISTCONTROL=ignoredups
# Set history size and file size
export HISTSIZE=1000
export HISTFILESIZE=2000
# Record timestamps with commands
export HISTTIMEFORMAT="%F %T "
```

After modifying the .bashrc, run source ~/.bashrc to apply the changes.

How to Use Ctrl + R

1. Interactive Reverse Search

When you press Ctrl + R, you enter the reverse search mode. This allows you to search through your command history starting from the most recent command and moving backward.

- Steps:
 - Press Ctrl + R to start the search.
 - Start typing part of a command you want to find. For example, type ls

if you want to find a previously used ls command.

- The shell will show the most recent command that matches the string you've typed so far.

Example:

- (reverse-i-search)`ls': ls -l /home/user/Documents

This means that the last command matching ls was ls -l /home/user/Documents.

2. Refining the Search

- Keep Searching Backward:
 - If the first match is not what you're looking for, press Ctrl + R again to continue searching backward for the next occurrence of the search term.
- Search Forward (if in Ctrl + R mode):
 - Press Ctrl + S to search forward in the history (though, on some systems, this may be bound to flow control, and you might need to disable that feature for it to work).

3. Execute the Found Command

Once you've found the command you want to use, you can do a few things:

- Execute the command:
 - Press Enter to execute the command directly.
- Edit the command:
 - Press Right Arrow or Left Arrow to move the cursor to the command line, where you can edit the command before running it.
- Cancel the search:
 - If you decide you don't want to use the command, press Ctrl + C to exit the search mode without executing anything.

Example of Ctrl + R in Action

- You want to find an earlier command that started with git.
 - Press Ctrl + R.
 - Start typing git. As you type, the shell will show the most recent git command.
 - (reverse-i-search)`git': git status
 - If this is the command you want to reuse, you can press Enter to run it again. If you want to find an older git command, keep pressing Ctrl + R until you find it.
- Once you've found the command, you might decide to edit it before executing. Press the Right Arrow to move the cursor to the command line, then make the necessary changes.

Ctrl + R and History Expansion

The reverse search will only find commands stored in your history, which is typically saved in the ~/.bash_history file (or whatever file you've configured in HISTFILE). If the command you're looking for isn't in the history, it won't appear in the search results.

Clearing the History Search

If you've gone through several commands and don't want to see old searches, or if you want to reset the search:

- Press Ctrl + C to cancel the search.
- If you're finished with your session and want to clear your history, you can use the history -c command.

File Globbing in Linux is a way of using wildcards to match multiple filenames or directories. It allows you to specify patterns that match a group of files or directories rather than specifying each file individually. This is commonly used in the shell for file manipulation, searching, and other operations.

Basic Wildcards in File Globbing

Here are the key wildcard characters you can use in file globbing:

1.1. Asterisk (*)

- Usage: The asterisk * matches zero or more characters in a filename or directory.

- Example:

- ls *.txt

This matches all files ending with .txt (e.g., file.txt, document.txt).

- Another Example:

- ls /home/user/*

This matches all files and directories under /home/user/.

1.2. Question Mark (?)

- Usage: The question mark ? matches exactly one character.

- Example:

- ls file?.txt

This matches file1.txt, fileA.txt, but not file.txt (because ? requires exactly one character after "file").

- Another Example:

- ls report-???.log

This matches files like report-01.log or report-AA.log, but not report-1.log or report-123.log.

1.3. Square Brackets ([])

- Usage: Square brackets allow you to match a range or specific set of characters.

- Range: Matches any one of a range of characters.
- Set: Matches any one of the listed characters inside the brackets.
- Examples:
 - Range:

- `ls file[1-3].txt`

This matches `file1.txt`, `file2.txt`, and `file3.txt`, but not `file4.txt`.

- Set:
- `ls file[a-c].txt`

This matches `filea.txt`, `fileb.txt`, and `filec.txt`, but not `filex.txt`.

- Negation (`[^]`):
- You can also use `[^]` to specify a set of characters to exclude.
- Example:

- `ls file[^a].txt`

This matches all files like `fileb.txt`, `filec.txt`, but not `filea.txt` (because of the `[^a]` negation).

1.4. Double Asterisk (*)

- Usage: The `*` (double asterisk) matches directories recursively, meaning it matches files in subdirectories as well.

- Example:
- `ls /*.txt`

This matches all `.txt` files in the current directory and all of its subdirectories.

- Another Example:
- `find . -name "*.txt"`

This finds all `.txt` files, including those in subdirectories.

2. Glob Patterns in Practice

Here are a few practical examples where globbing is helpful:

2.1. Copy Files Using Wildcards

Copy all `.txt` files from one directory to another:

```
cp /path/to/files/*.txt /destination/directory/
```

2.2. Remove Files

Delete all `.log` files in the current directory:

```
rm *.log
```

2.3. List Files Starting with a Specific Letter

List all files starting with the letter `a`:

```
ls a*
```

2.4. Searching for Specific Files

Search for all `.jpg` and `.jpeg` files in the `/images` directory:

```
ls /images/*.{jpg,jpeg}
```

2.5. Working with Multiple Extensions

If you want to list all files that either have `.jpg`, `.jpeg`, or `.png` extensions:

```
ls *.{jpg,jpeg,png}
```

3. Escaping Special Characters

Sometimes, you may want to literally match special characters like `*`, `?`, or `[]`. To do this, you can escape them with a backslash (`\`).

- Example:
- `ls file\*.txt`

This will match a file literally named `file*.txt` rather than interpreting the `*` as a wildcard.

- You can also use quotes (`'` or `"`) to avoid expanding special characters in a glob pattern.

- `ls 'file*.txt'`

4. Advanced Globbing

In addition to simple globbing, there are advanced features you can use in the shell.

4.1. Extended Globbing (shopt in Bash)

Bash allows extended globbing (advanced pattern matching) when you enable the `extglob` option. This enables more complex pattern matching.

- Enable Extended Globbing:
 - `shopt -s extglob`
- Patterns:
 - `@(pattern1|pattern2)`: Matches one of the patterns.
 - `!(pattern)`: Matches anything except the given pattern.
 - `?(pattern)`: Matches zero or one occurrence of the pattern.
 - `*(pattern)`: Matches zero or more occurrences of the pattern.
 - `+(pattern)`: Matches one or more occurrences of the pattern.
- Examples:
 - Match files that are either `file1.txt` or `file2.txt`:
 - `ls @(file1.txt|file2.txt)`
 - Match files that do not end with `.txt`:
 - `ls !(*.txt)`
 - Match all files that start with `a`, `b`, or `c`:
 - `ls [abc]*`

5. Summary of Common Globbing Patterns

Pattern	Description	Example
<code>*</code>	Matches zero or more characters	<code>*.txt</code> (all <code>.txt</code> files)
<code>?</code>	Matches exactly one character	<code>file?.txt</code> (matches <code>file1.txt</code> , <code>fileA.txt</code>)
<code>[]</code>	Matches any one of the characters in the brackets	<code>file[1-3].txt</code> (matches <code>file1.txt</code> , <code>file2.txt</code> , <code>file3.txt</code>)
<code>[^]</code>	Matches any one character not in the brackets	<code>file[^a].txt</code> (matches any file except <code>filea.txt</code>)
<code>*/</code>	Matches directories recursively	<code>/*.txt</code> (matches <code>.txt</code> files in subdirectories)
<code>*.{jpg,jpeg,png}</code>	Matches files with specific extensions	<code>*.{jpg,jpeg,png}</code> (matches <code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> files)

I/O Redirections

Input/Output (I/O) redirection in Linux refers to the ability of the Linux operating system that allows us to change the standard input (`stdin`) and standard output (`stdout`) when executing a command on the terminal.

By default, the standard input device is your keyboard and the standard output device is your screen.

Using Redirectors, we can send any output to files or receive input from files. Redirectors come in very handy when we work with large files or outputs. For Example, You want to get the list of all files in a directory which are modified 1 day ago. You can use the `"find"` command to list the files and store the list using redirectors in another file for later analysis, debugging or sharing with others.

Streams

In the Linux environment, input and output are distributed across three streams. The streams are as follows:

- Standard Input (`stdin`): The standard input source, which is usually the keyboard.
- Standard Output (`stdout`): The terminal is often the default output destination.
- Standard Error (`stderr`): The stream for error messages, which is also by default sent to the terminal.

These are also numbered as :

- 0 (`stdin`)
- 1 (`stdout`)
- 2 (`stderr`)

Redirecting Operators

Overwrite: These Operators overwrite the existing content of the file.

- `>` - standard output
- `<` - standard input

- 2 > - standard error

Append: Double bracket operators append the content into the existing content of the file.

- >> - standard output
- << - standard input
- 2>> - standard error

Types of I/O Redirection

1. Redirecting Output (stdout)

The most common form of redirection is redirecting output from the default screen (stdout) to a file or other destination.

To a file:

To redirect the standard output to a file, use the > operator:

command > file.txt

- Example:
- echo "Hello, World!" > output.txt

This will create (or overwrite) a file output.txt containing the text Hello, World!.

Appending to a file:

If you want to append output to a file instead of overwriting it, use the >> operator:

command >> file.txt

- Example:
- echo "Another line" >> output.txt

This will add the string "Another line" to the end of output.txt without erasing its existing contents.

2. Redirecting Input (stdin)

You can redirect input to a command from a file using the < operator. This allows a command to receive its input from a file rather than the keyboard.

command < file.txt

- Example:
- cat < input.txt

This will display the contents of input.txt.

3. Redirecting Both Output and Error

You can redirect both stdout and stderr to the same file.

command > file.txt 2>&1

- Explanation:
 - 2 refers to the file descriptor for stderr.
 - >&1 means redirect stderr (2) to the same location as stdout (1).
- Example:
- ls /nonexistent_directory > output.txt 2>&1

This will write both the standard output and any error messages to output.txt.

4. Redirecting Error (stderr)

You can redirect only the error output (stderr) to a file using 2>:

command 2> error.txt

- Example:
- ls /nonexistent_directory 2> error.txt

This will store the error message in error.txt without affecting the normal output.

5. Pipes (|)

A pipe is a method of redirecting the output of one command to the input of another. It allows the output of one program to be used as input for another, creating a "pipeline" of commands.

command1 | command2

- Example:
- cat file.txt | grep "Hello"

This will search for the word "Hello" in file.txt by passing the file's contents to the grep command through the pipe.

6. Here Documents (<<)

A Here Document is used to provide multiline input to a command. It allows you to pass input directly within a script or command line, rather than from a file.

command << EOF

line 1

line 2

line 3

EOF

- Example:
- cat << EOF

This is the first line.

This is the second line.

EOF

This will output:

This is the first line.

This is the second line.

7. Here Strings (<<<)

A Here String is a variant of Here Document that allows you to pass a single string of data directly to a command.

command <<< "string_data"

- Example:
- grep "Hello" <<< "Hello, World!"

This will search for "Hello" in the string "Hello, World!".

8. File Descriptors

In Linux, everything is treated as a file, and file descriptors are used to refer to open files or streams. By default:

- 0 is the file descriptor for stdin.
- 1 is for stdout.
- 2 is for stderr.

You can redirect to and from specific file descriptors using the syntax:

command 3> file.txt

This directs file descriptor 3 to file.txt. This is a more advanced form of redirection used for handling multiple file streams.

Summary of Key Operators

- >: Redirects standard output to a file (overwrites).
- >>: Redirects standard output to a file (appends).
- <: Redirects standard input from a file.
- 2>: Redirects standard error to a file.
- 2>&1: Redirects standard error to standard output.
- |: Pipe, redirects the output of one command to the input of another.
- <<: Here Document, allows multiline input to a command.
- <<<: Here String, allows single-line input to a command.

Command Description

">" Used to redirect output to a file, creating a new file if it doesn't exist or overwriting the existing file if it does.

">>" Used to redirect output to a file, but appending the output to the end of the file instead of overwriting it.

"<" Used to redirect input from a file, with the contents of the file being used as input to the command.

"<<" Used to redirect input from a "here document", which is a way to pass input to a command without using a file.

"&>" Used to redirect both standard output and standard error to a file, creating a new file if it doesn't exist or overwriting the existing file if it does.

"&>>" Used to redirect both standard output and standard error to a file, but appending the output to the end of the file instead of overwriting it.

"2>" Used to redirect standard error to a file, creating a new file if it doesn't exist or overwriting the existing file if it does.

"2>>" Used to redirect standard error to a file, but appending the output to the end of the file instead of overwriting it.

"|" Used to redirect the output of one command as input to another command, creating a pipeline of commands.

"tee" Used to redirect output to a file, while still displaying the output on the screen.

"<<<" Used for here strings, which allow you to specify input directly in a variable.

"<>" Used to open a file for both reading and writing.

">&" Used to redirect standard output and standard error to a file descriptor.

"<()" Used for command substitution, which allows the output of a command to be used as input for another command.

">()" Used for process substitution, which allows the output of a command to be used as input for another command as a file.

"script" Used to record a shell session to a file for later replay.

"/dev/null" Used to discard output, effectively sending it into a "black hole".

Redirection operators in Linux

So far in the article, we have come to the understanding that I/O redirection allows the user to change the input source of a command and the destination of the command output by using the redirection operators "<" and ">". In Linux, you can use various commands and operators to redirect the input, output, and errors between the command line and files or other processes, which you will learn about these operators in the following:

- "> ": Standard Output Redirection and Overwrites the file's content
- ">>": redirecting and Appending Standard Output
- "<": Standard input redirection of command from a file rather than the keyboard
- "2>": Redirects the Standard error of a command to a file and overwrites the file's contents.
- "2>>": Redirects the Standard error of a command to a file and appends the error to the file
- "<<<": Standard input redirection from the block of text rather than command
- "&>" or "2>&1": Redirecting Standard Output and Error to the same file
- "&>>": Appending Standard Output and Error to the specified file
- "<&": redirects input of one file to another.
- | (Piping): Sends the standard output of one command as the standard input to another command.
- "<<<<": uses a string as the input to a command.
- "<>": Specifies the files to which standard input and errors are directed

As a result, by learning the redirection operators in Linux, you can control and manage the input and output flows in different Linux distributions such as Debian, Ubuntu, CentOS, Fedora, etc., and also, redirection operators allow you to manipulate and manage data to optimize workflow.

3. input Redirection in Linux

In normal mode, a program reads the input through the command that the user types on the keyboard, but what should we do if we don't want the input to be a command connected to the keyboard and the program receives the input from a file?

The symbol (<) is used to redirect standard input from a file instead of the keyboard; that is, by using this operator, the program reads input from a file instead of reading from the keyboard. The basic syntax to redirect the standard input from the file instead of the keyboard is as follows:

command < file

For example:

less < /etc/passwd

In the previous example, using the (<) operator, the command's standard input is taken from the /etc/passwd file instead of the keyboard.

Note: The "<" symbol can also be used instead of (<) because they have the same function.

In addition to the (<), you can use the (<<) operator to redirect the standard input without overwriting the content of the target file if it existed before. The (<<) operator allows the user to specify a text block as input to a command and appends it to the file's contents.

For example:

```
command << DELIMITER
Text input
Goes here
DELIMITER
```

To pass multiple input lines to one command, you can use the (<<) as follows:

```
command << END
This is line 1
This is line 2
END
```

Between << END and END, you can send multiple input lines to the command.

Also, the (<<<) operator allows using string as input for a command:

```
command <<< "Input String"
```

4. output Redirection in Linux

Operators (> and >>) are used to redirect the standard output of a command to a file so that the output of this command is used as the standard input of another command. You may ask that If we want to redirect the standard output to a different file, should we use ">" or ">>"? In the following, we describe when you should use ">" and ">>".

(>) operator: If you use the (>) to redirect the standard output to a file, the content of the file will be overwritten if it exists:

```
command > output_file
```

For example:

```
ls ~ > root_dir_contents.txt
```

By running the previous command, you will not see the output in the terminal because the output of the above command is sent to the root_dir_contents.txt file, and if this file already has contents, its content will be deleted, and the output of the previous command will be written in it because the (>)operator overwrites the content of the file.

Note: If you want to delete the command output, you can redirect the standard output of a command to the /dev/null file; Because /dev/null is a special file, any data sent to the /dev/null file will be deleted. Therefore, to discard standard output that is unused, you can also use the following command:

```
command > /dev/null
```

It is worth noting that /dev/null, also known as "the black hole," you should be careful in using the /dev/null file in your commands because it can have dangerous consequences. For example, the "mv folder /dev/null" command, one of the most dangerous Linux commands, is sometimes used to transfer files to files, which causes the deletion of your important data.

(>>) operator: This symbol is also used to redirect standard output to a file, but instead of overwriting the content of the specified file (if it already exists), it appends the standard output of the command at the end of the target file:

```
command >> output_file
```

For example:

```
sudo apt-cache pkgnames >> pkgnames
```

In this example, the output of the command is sent to the pkgnames file, and if the pkgnames file already exists, the output of the command is appended to the end of the pkgnames file, but the contents of the pkgnames file will not be overwritten.

Note: Instead of the (>) operator, you can use (1>) to redirect the standard output.

5. Standard Error Redirection in Linux

The (2> and 2>>) operators allow you to redirect the standard error of a command to a file.

"2>" operator: redirects the standard error of a command to the target file; if the target file already exists, it overwrites the file's contents. Use the following basic syntax to redirect a command's standard error to a file:
command 2> file

For example:

```
ls -l /root/ 2>ls-error.log
```

If you run the command without root permission, you will encounter an error; with the previous command, you sent the error of the ls command to the ls-error.log file, and if the ls-error.log file already exists, by running the "2>" operator the content of the file is overwritten.

Note that the error message is displayed as text in the terminal but is also sent to the target file.

"2>>" operator: "2>>" redirects the Standard error of a command to the file, but if the file exists, it will not overwrite its contents but appends the error to the end of the specified file. As a result, if you do not want the content of the error file to be deleted and overwritten, and you want to have an error report file for a service whose content will not be deleted by writing to it, we recommend using the "2>>" operator to redirect the standard error.

6. Standard output/error redirection to a file

"&>" or "2>&1" are efficient operators that are used to redirect the standard output and error of a command to a single file simultaneously. The main syntax for simultaneously redirecting the output and error message of a command to a single file is as follows:

```
command &> file
```

Or

```
command 2>&1 file
```

If you want output and standard error to be appended to a single file, you can use "&>>" instead of "&>" or "2>&1":
command &>> file

7. Combining Input and Output Redirection

Fortunately, Linux allows you to combine input and output redirection in one command. To make the command take input from input_file and direct output to output_file, run the following syntax:
command input_file output_file

For example:

```
sort domains.list sort.output
```

In this example, the sort command takes the input from the domains.list file and passes it to the sort.output file.

8. input /output redirection in linux using pipes

pipes (|) is the best way to combine multiple commands. Because pipes allow you to use the output of one command as the input of another command:

```
command1 | command2
```

This pattern is executed with the purpose that the standard output of command 1 is redirected to the standard input of command 2.

For example:

```
ls | less
```

By running the previous command, the output of the ls command, which presents the contents of your current directory, is used as the input of the less command, and the less command displays the contents of your current directory

one line at a time. As a result, the contents of your current directory will be displayed as each entry in a new line by running the previous command.

9. Redirecting output to multiple files

Using the tee command, you can redirect the standard output to several files and simultaneously receive the command output in the terminal. To redirect the standard output of a command to a file at the same time you see it in the Linux terminal, use the following syntax:

```
command | tee output_file1 output_file2
```

This causes the output of the command to be redirected to another file in addition to being displayed in the Linux terminal, and if the file already exists, the file's content will be overwritten; if the file does not already exist, a new file will be created. For example:

```
wc /etc/magic | tee magic_count.txt
```

In this example, the output of the "wc" command is sent in two places: 1. Terminal 2. The magic_count.txt file.

Text processing utilities

Here is a complete list of common Linux text filters, along with their descriptions and examples.

1. cat (Concatenate and Display Files)

The cat command is used to display, concatenate, or create files.

Example:

```
cat file.txt # Display the contents of file.txt
```

```
cat file1.txt file2.txt > combined.txt # Concatenate file1.txt and file2.txt  
into combined.txt
```

2. tac (Concatenate and Display in Reverse)

The tac command is like cat, but it displays the contents of files in reverse order (line by line).

Example:

```
tac file.txt # Display file.txt with lines reversed (from bottom to top)
```

3. nl (Number Lines of Files)

The nl command is used to number the lines in a file.

Example:

```
nl file.txt # Display file.txt with line numbers
```

4. more (View File Contents)

The more command is used to view the contents of a file one screen at a time.

Example:

```
more file.txt # View file.txt one screen at a time
```

5. less (View File Contents with Navigation)

The less command is similar to more but allows backward navigation. It is a more advanced file viewer.

Example:

```
less file.txt # View file.txt with scrolling capabilities (up and down)
```

6. head (View the First Part of Files)

The head command displays the first n lines of a file (default is 10).

Example:

```
head file.txt # Display the first 10 lines of file.txt
```

```
head -n 20 file.txt # Display the first 20 lines of file.txt
```

7. tail (View the Last Part of Files)

The tail command displays the last n lines of a file (default is 10). It is commonly used to view log files.

Example:

```
tail file.txt # Display the last 10 lines of file.txt
tail -n 20 file.txt # Display the last 20 lines of file.txt
tail -f /var/log/syslog # Follow the file as it grows (useful for log files)
```

8. cut (Remove Sections of Each Line)

The cut command is used to extract sections from each line of input based on delimiters, byte positions, or field numbers.

Example:

```
cut -d ':' -f 1,3 /etc/passwd # Extract fields 1 and 3 from /etc/passwd (using ':' delimiter)
```

9. paste (Merge Lines of Files)

The paste command merges lines of files horizontally (side by side).

Example:

```
paste file1.txt file2.txt # Merge the contents of file1.txt and file2.txt side by side
```

10. sort (Sort Lines of Text)

The sort command is used to sort lines in a file or input stream.

Example:

```
sort file.txt # Sort file.txt in alphabetical order
sort -n file.txt # Sort numerically
sort -r file.txt # Sort in reverse order
```

11. uniq (Report or Omit Repeated Lines)

The uniq command removes duplicate lines from a file (after sorting).

Example:

```
sort file.txt | uniq # Sort and remove duplicate lines
uniq file.txt # Remove duplicates from an already sorted file
```

12. wc (Word, Line, and Character Count)

The wc command counts the number of lines, words, and characters in a file or input.

Example:

```
wc file.txt # Count lines, words, and characters in file.txt
wc -l file.txt # Count lines only
wc -w file.txt # Count words only
wc -c file.txt # Count characters only
```

13. grep (Search for Text)

The grep command searches for text patterns in files or input. It supports regular expressions.

Example:

```
grep "pattern" file.txt # Search for "pattern" in file.txt
grep -i "pattern" file.txt # Case-insensitive search
grep -r "pattern" /path/to/dir # Search recursively in a directory
```

14. egrep (Extended Grep, Supports Extended Regular Expressions)

egrep is a variant of grep that supports extended regular expressions.

Example:

```
egrep "pattern1|pattern2" file.txt # Search for either pattern1 or pattern2
```

15. sed (Stream Editor)

The sed command is a stream editor used to perform basic text transformations (such as substitution, insertion, deletion) on an input stream or file.

Example:

```
sed 's/old/new/' file.txt # Replace first occurrence of "old" with "new" in each line
sed 's/old/new/g' file.txt # Replace all occurrences of "old" with "new" in each line
sed -i 's/old/new/' file.txt # Modify the file in place (no output)
```

16. awk (Pattern Scanning and Processing Language)

The awk command is a powerful text-processing language used for pattern scanning and processing.

Example:

```
awk '{print $1}' file.txt # Print the first column of file.txt (by default,
whitespace delimited)
awk -F: '{print $1, $3}' /etc/passwd # Print the first and third fields from
/etc/passwd (colon-separated)
```

17. tr (Translate or Delete Characters)

The tr command is used to translate or delete characters from input.

Example:

```
echo "hello" | tr 'a-z' 'A-Z' # Convert lowercase to uppercase
echo "hello" | tr -d 'l' # Delete all occurrences of 'l'
```

18. find (Search for Files in a Directory Hierarchy)

The find command searches for files and directories in a directory hierarchy.

Example:

```
find /path/to/dir -name "*.txt" # Find all .txt files in /path/to/dir
find /path/to/dir -type f -size +100M # Find files larger than 100MB
```

19. xargs (Build and Execute Command Lines from Input)

The xargs command is used to build and execute command lines from standard input.

Example:

```
echo "file1 file2 file3" | xargs rm # Delete files file1, file2, file3
find . -name "*.log" | xargs rm # Find and delete all .log files in the current
directory
```

20. tee (Read from Standard Input and Write to Standard Output and Files)

The tee command reads from standard input and writes to both standard output and one or more files.

Example:

```
echo "hello" | tee file.txt # Display "hello" and write it to file.txt
echo "hello" | tee -a file.txt # Append "hello" to file.txt and display it
```

21. diff (Compare Files Line by Line)

The diff command compares files line by line and outputs the differences.

Example:

```
diff file1.txt file2.txt # Compare file1.txt with file2.txt
```

22. cmp (Compare Two Files Byte by Byte)

The cmp command compares two files byte by byte.

Example:

```
cmp file1.txt file2.txt # Compare file1.txt and file2.txt byte by byte
```

23. join (Join Lines of Two Files on a Common Field)

The join command combines lines from two files based on a common field (by default, the first field).

Example:

```
join file1.txt file2.txt # Join file1.txt and file2.txt on the first field
```

24. wc (Word, Line, Character, Byte Count)

The wc command counts the number of lines, words, characters, and bytes in a file.

Example:

```
wc file.txt # Count lines, words, and characters in file.txt
```

25. split (Split a File into Pieces)

The split command splits a file into smaller files.

Example:

```
split -l 1000 largefile.txt # Split largefile.txt into files with 1000 lines
each
```

26. fmt (Simple Text Formatter)

The fmt command formats text to fit within a specified line length.

Example:

`fmt file.txt # Format file.txt to fit lines to the terminal width`

27. **shuf** (Generate Random Permutations)

The `shuf` command shuffles the input and prints random permutations.

Example:

`shuf file.txt # Shuffle the lines of file.txt`

28. **column** (Columnate Text Output)

The `column` command formats text into columns.

Example:

`column -t file.txt # Format the contents of file.txt into neat columns`

29. **sed** (Stream Editor)

The `sed` command is a powerful stream editor for performing basic text transformations.

Example:

`sed 's/old/new/' file.txt # Replace "old" with "new" in file.txt`

30. **perl** (Practical Extraction and Reporting Language)

`perl` can be used for advanced text manipulation.

Example:

`perl -pe 's/old/new/' file.txt # Replace "old" with "new" using Perl`

31. **uniq** (Report or Omit Repeated Lines)

The `uniq` command filters out repeated lines in a file or stream. However, it only removes consecutive duplicate lines, so it is usually used in combination with `sort` to eliminate all duplicates from a file or input stream.

Usage:

`uniq [options] [input_file] [output_file]`

Common Options:

- `-u`: Only show unique lines (lines that appear only once).
- `-d`: Only show duplicate lines (lines that appear more than once).
- `-c`: Prefix each line with the number of occurrences.
- `-i`: Ignore case while comparing lines.
- `-w N`: Compare only the first N characters of each line.

Examples:

- Remove Duplicate Lines (Consecutive):
 - `sort file.txt | uniq`
 - The `sort` command ensures that all duplicate lines are adjacent, and `uniq` then removes them.
- Count Occurrences of Each Line:
 - `sort file.txt | uniq -c`
 - This will show how many times each line occurs in the file.
- Show Only Duplicate Lines:
 - `sort file.txt | uniq -d`
 - This will display only the lines that are repeated in `file.txt`.
- Show Only Unique Lines:
 - `sort file.txt | uniq -u`
 - This will display lines that appear only once.

32. **comm** (Compare Two Sorted Files Line by Line)

The `comm` command compares two sorted files line by line and produces three columns of output:

- Lines only in the first file.
- Lines only in the second file.
- Lines common to both files.

To use `comm`, both files must be sorted beforehand.

Usage:

`comm [options] file1 file2`

Common Options:

- `-1`: Suppress the output of lines that are only in `file1`.
- `-2`: Suppress the output of lines that are only in `file2`.
- `-3`: Suppress the output of lines that are in both files.

Examples:

- Compare Two Files:
- `comm file1.txt file2.txt`
- This will display three columns:
 - Column 1: Lines unique to file1.txt.
 - Column 2: Lines unique to file2.txt.
 - Column 3: Lines common to both files.
- Suppress Common Lines:
- `comm -3 file1.txt file2.txt`
- This will display only the lines that are unique to each file, omitting the common lines.
- Show Only Lines in file1 and Not in file2:
- `comm -2 file1.txt file2.txt`
 - This will suppress the lines unique to file2.txt and the common lines, showing only those in file1.txt.

33. **od** (Octal Dump)

The `od` (Octal Dump) command is used to display the contents of a file in different formats such as octal, hexadecimal, ASCII, etc. This command is very useful when you need to inspect non-printable characters, binary files, or when you need to analyze the raw byte contents of a file.

Usage:

`od [options] [file]`

Common Options:

- `-c`: Display output as characters (ASCII), showing the control characters in a readable form.
- `-b`: Display output in octal format.
- `-x`: Display output in hexadecimal format.
- `-t`: Display in various formats (e.g., `-t x1` for hexadecimal, `-t c1` for characters).
- `-A`: Specify the address format (e.g., `-A d` for decimal, `-A x` for hexadecimal).
- `-v`: Output all data (no compression of identical lines).

Examples:

- Display Contents in Octal:
- `od -b file.txt`
- This will display the contents of file.txt in octal format, where each byte is represented by its octal value.
- Display Contents in Hexadecimal:
- `od -x file.txt`
- This will display the contents of file.txt in hexadecimal format.
- Display Contents as Characters (ASCII):
- `od -c file.txt`
- This will display the file contents with non-printable characters replaced by escape sequences, making them readable.
- Display All Data (no compression of identical lines):
- `od -v file.txt`
- This will display the entire file content, showing every line, even if they are identical.
- Display Hexadecimal Output with Addresses:
- `od -A x -t x1 file.txt`
- This will display the content of file.txt in hexadecimal format, with the addresses in hexadecimal as well.
- Combine Different Output Formats:
- `od -c -x file.txt`
- This will show both the ASCII and hexadecimal representations of file.txt in the output.

Archiving and Compression

Key Concepts:

- Archiving: The process of collecting files into a single archive file (often with a `.tar` extension) without necessarily compressing them.

- **Compression:** The process of reducing the file size using compression algorithms to save disk space or make file transfer faster (typically creating .gz, .bz2, .xz, or .zip extensions).

- **Combination:** It's common to combine archiving and compression,

e.g., .tar.gz or .tar.bz2 files, to both group files and reduce their size.

Archiving Files in Linux

1.1. tar (Tape Archive)

The most widely used tool for creating archives is tar. It combines multiple files into one file, but it does not compress the files by default.

Basic Syntax:

```
tar [options] archive_name file1 file2 ...
```

Common Options:

- -c: Create a new archive.
- -x: Extract files from an archive.
- -f: Specify the archive file name.
- -v: Verbose output (lists files being processed).
- -t: List the contents of an archive.
- -C: Change to a specific directory before extracting.
- -z: Compress the archive using gzip.
- -j: Compress the archive using bzip2.
- -J: Compress the archive using xz.
- -p: Preserve file permissions.

Examples:

- Create an Archive:
 - `tar -cvf archive.tar file1.txt file2.txt`
- -c: Create a new archive.
- -v: Verbose mode (lists files).
- -f archive.tar: Specifies the archive file name.
- Extract an Archive:
 - `tar -xvf archive.tar`
- -x: Extract the archive.
- List Contents of an Archive:
 - `tar -tvf archive.tar`
- -t: List contents.
- Create a Compressed Archive (gzip):
 - `tar -czvf archive.tar.gz file1.txt file2.txt`
- -z: Compress using gzip.
- Extract a Compressed Archive (gzip):
 - `tar -xzvf archive.tar.gz`
- Create a Compressed Archive (bzip2):
 - `tar -cjvf archive.tar.bz2 file1.txt file2.txt`
- -j: Compress using bzip2.
- Extract a Compressed Archive (bzip2):
 - `tar -xjvf archive.tar.bz2`
- Create a Compressed Archive (xz):
 - `tar -cJvf archive.tar.xz file1.txt file2.txt`
- -J: Compress using xz.
- Extract a Compressed Archive (xz):

```
tar -xJvf archive.tar.xz
```

Compression Utilities

2.1. gzip (GNU Zip)

gzip is one of the most common compression tools in Linux. It is typically used to compress individual files.

Basic Syntax:

```
gzip [options] file
```

Common Options:

- -d: Decompress.
- -c: Write output to stdout (useful for piping).
- -v: Verbose mode (shows compression ratio).

- -r: Recursively compress files in a directory.
- -9: Maximum compression (slowest).
- -1: Fastest compression (least effective).

Examples:

- Compress a File:
- `gzip file.txt`
- This will create a compressed file `file.txt.gz`.
- Decompress a File:
- `gzip -d file.txt.gz`
- Compress with Maximum Compression:
- `gzip -9 file.txt`
- Compress and Keep the Original File:
- `gzip -c file.txt > file.txt.gz`

2.2. bzip2

bzip2 offers better compression ratios than gzip, though it is generally slower.

Basic Syntax:

`bzip2 [options] file`

Common Options:

- -d: Decompress.
- -k: Keep the original file.
- -z: Compress (default operation).
- -1 to -9: Compression levels (1 is fastest, 9 is slowest).
- -v: Verbose output.

Examples:

- Compress a File:
- `bzip2 file.txt`
- This will create `file.txt.bz2`.
- Decompress a File:
- `bzip2 -d file.txt.bz2`
- Compress with Maximum Compression:
- `bzip2 -9 file.txt`
- Compress and Keep the Original File:
- `bzip2 -k file.txt`

2.3. xz

xz provides excellent compression ratios, often surpassing gzip and bzip2, but it's slower. It is widely used in package management systems (e.g., `.tar.xz` archives).

Basic Syntax:

`xz [options] file`

Common Options:

- -d: Decompress.
- -z: Compress (default).
- -k: Keep the original file.
- -9: Maximum compression.

Examples:

- Compress a File:
- `xz file.txt`
- This creates a file `file.txt.xz`.
- Decompress a File:
- `xz -d file.txt.xz`
- Compress with Maximum Compression:
- `xz -9 file.txt`
- Compress and Keep the Original File:
- `xz -k file.txt`

2.4. zip (ZIP Compression)

zip is often used in Linux and is more compatible with other operating systems, especially Windows. It compresses files and directories into `.zip` archives.

Basic Syntax:

`zip [options] archive_name.zip file1 file2 ...`

Common Options:

- -r: Recursively zip files in directories.

- -d: Delete entries from a zip archive.
- -e: Encrypt the zip file.
- -x: Exclude files matching a pattern.

Examples:

- Create a ZIP Archive:
- `zip archive.zip file1.txt file2.txt`
- Create a ZIP Archive Recursively (For Directories):
- `zip -r archive.zip folder1 folder2`
- Decompress a ZIP File:
- `unzip archive.zip`
- List Contents of a ZIP File:
- `unzip -l archive.zip`
- Extract Files to a Specific Directory:

`unzip archive.zip -d /path/to/directory`

Combining Archiving and Compression

In Linux, it is common to combine archiving and compression using tar along with compression tools like gzip, bzip2, or xz.

Examples:

- Create a tar.gz (GZIP) Archive:
- `tar -czvf archive.tar.gz file1.txt file2.txt`
- Create a tar.bz2 (BZIP2) Archive:
- `tar -cjvf archive.tar.bz2 file1.txt file2.txt`
- Create a tar.xz (XZ) Archive:
- `tar -cJvf archive.tar.xz file1.txt file2.txt`
- Extract from tar.gz:
- `tar -xzf archive.tar.gz`
- Extract from tar.bz2:
- `tar -xjf archive.tar.bz2`

4. Comparison of Compression Tools

Tool	File Extension	Compression Ratio	Speed	Use Case
gzip	.gz	Moderate	Fast	Standard compression for single files.
bzip2	.bz2	High	Slow	Better compression ratio than gzip.
xz	.xz	Very High	Very Slow	Best compression ratio, used for packages.
zip	.zip	Moderate	Moderate	Cross-platform archives, used for folders.
tar	.tar	None	None	Used for archiving (not compression).

5. Conclusion

Linux provides several powerful tools for archiving and compression, each suited for different needs. Here's a quick rundown:

- tar: The go-to tool for archiving files and directories. It can be used with compression tools like gzip, bzip2, and xz.
- gzip: Fast compression with moderate efficiency. Commonly used for web or network transfers.
- bzip2: Better compression than gzip but slower.
- xz: Best compression ratio but slow, ideal for packaging.
- zip: Common in cross-platform environments, useful for archiving folders with compression.

Introduction to Vim

Vim is an advanced version of Vi, a text editor that comes pre-installed on most Unix-like operating systems. Vim is designed to be used entirely with the keyboard, and its modes give it powerful text-editing capabilities.

Key Vim Modes:

- Normal Mode (default mode when you open Vim) - For navigation and text manipulation.
- Insert Mode - For typing and editing text.
- Visual Mode - For selecting text.

- Command Mode - For issuing commands (e.g., saving, quitting).

2. Starting and Exiting Vim

Starting Vim:

- Open a file with Vim:
 - `vim filename.txt`
 - This opens `filename.txt` in Vim.
- If the file doesn't exist, Vim will create a new file with that name when you save.

Exiting Vim:

- To exit Vim, press Esc to make sure you're in Normal Mode, then:
 - Save and quit:
- `:wq`
- Quit without saving:
- `:q!`
- Save without quitting:
 - `:W`

3. Understanding Vim Modes

Normal Mode (Default Mode)

- Navigation:
 - Use h, j, k, l to move left, down, up, and right.
 - w: Move to the beginning of the next word.
 - b: Move to the beginning of the previous word.
 - 0: Move to the beginning of the current line.
 - \$: Move to the end of the current line.
 - gg: Go to the top of the file.
 - G: Go to the end of the file.
- Deleting Text:
 - x: Delete the character under the cursor.
 - dd: Delete the current line.
 - d\$: Delete from the cursor position to the end of the line.
 - d0: Delete from the cursor position to the beginning of the line.
- Copying/Pasting:
 - yy: Copy (yank) the current line.
 - p: Paste after the cursor.
 - P: Paste before the cursor.
- Undo/Redo:
 - u: Undo the last action.
 - Ctrl-r: Redo the last undone action.
- Searching:
 - /pattern: Search for a pattern forward in the file.
 - ?pattern: Search backward.
 - n: Repeat the search in the same direction.
 - N: Repeat the search in the opposite direction.
- Replacing:
 - :s/old/new/: Replace the first occurrence of "old" with "new" on the current line.
 - :s/old/new/g: Replace all occurrences on the current line.
 - :%s/old/new/g: Replace all occurrences in the entire file.

Insert Mode

- Entering Insert Mode: Press i to start typing at the current cursor position.
- Other ways to enter Insert Mode:
 - I: Insert at the beginning of the current line.
 - a: Append after the cursor.
 - A: Append at the end of the current line.
 - o: Open a new line below the current line.
 - O: Open a new line above the current line.
- Exiting Insert Mode: Press Esc to return to Normal Mode.

Visual Mode

- Entering Visual Mode: Press v to start selecting text.

- V: Select an entire line.
- Ctrl-v: Enter Visual Block mode for column-wise selection.
- Copying and Pasting:
 - y: Yank (copy) the selected text.
 - d: Delete the selected text.
 - p: Paste after the cursor.
 - P: Paste before the cursor.

Command Mode

- Entering Command Mode: Press : from Normal Mode to start typing a command.
- Basic Commands:
 - :w: Save the file.
 - :q: Quit Vim (only if no changes are made).
 - :wq: Save and quit.
 - :q!: Quit without saving.
 - :x: Same as :wq.

4. Advanced Vim Commands and Features

Navigation Enhancements

- Ctrl-f: Move forward one screen.
- Ctrl-b: Move backward one screen.
- Ctrl-d: Scroll down half a screen.
- Ctrl-u: Scroll up half a screen.

Search Enhancements

- *: Search for the word under the cursor (forward).
- #: Search for the word under the cursor (backward).
- n: Repeat search forward.
- N: Repeat search backward.

Find and Replace

- :%s/old/new/gc: Replace all occurrences of "old" with "new" in the entire file, and prompt for confirmation for each replacement.

Buffers and Windows

- Ctrl-w + s: Split the window horizontally.
- Ctrl-w + v: Split the window vertically.
- :e filename: Open a new file in the current window.
- :bnext: Go to the next buffer (open file).
- :bprev: Go to the previous buffer.
- :bd: Delete (close) the current buffer.

Tabs

- :tabnew: Open a new tab.
- :tabnext: Go to the next tab.
- :tabprev: Go to the previous tab.

5. Customizing Vim

Vim is highly customizable via its configuration file, ~/.vimrc (or ~/.vim/vimrc). Some basic customizations include:

Setting Line Numbers

```
set number " Show line numbers
set relativenumber " Show relative line numbers
```

Enable Line Wrapping

```
set wrap
```

Set Indentation

```
set tabstop=4 " Set tab width to 4 spaces
set shiftwidth=4 " Set indentation width to 4 spaces
set expandtab " Use spaces instead of tabs
```

Enable Syntax Highlighting

```
syntax enable
```

Search Highlighting

```
set hlsearch " Highlight search results
set incsearch " Incremental search
```

Setting a Custom Color Scheme

```
colorscheme desert " Set a color scheme (e.g., "desert")
```

Searching in Vim

There are several ways to search for text in Vim, from basic searches to advanced pattern matching using regular expressions.

Basic Search:

- **Forward Search:**

- Press `/` followed by the search term and press Enter to search forward (downwards in the file).

- Example: `/hello` will search for the word "hello" from the current cursor position to the end of the file.

- **Backward Search:**

- Press `?` followed by the search term and press Enter to search backward (upwards in the file).

- Example: `?hello` will search for the word "hello" from the current cursor position to the beginning of the file.

Search Shortcuts:

- `n`: Repeat the last search in the same direction (forward if you searched forward, backward if you searched backward).

- `N`: Repeat the last search in the opposite direction (reverse direction of the previous search).

Case Sensitivity in Search:

- By default, searches are case-sensitive.

- Example: `/Hello` will only match "Hello" with an uppercase "H", not "hello".

- To make the search case-insensitive, use:

- `/hello\c`: Search for "hello" in a case-insensitive manner.

- Alternatively, you can toggle case-insensitive search for the entire session with:

- `:set ignorecase`

- If you want to override the case-insensitivity for a specific search, use:

- `/hello\C`: Case-sensitive search for "hello".

Highlighting Search Results:

- `set hlsearch`: This option will highlight all matches in the file as you type your search.

- `set nohlsearch`: Use this command to turn off search highlighting.

- You can quickly remove highlighting after a search by typing:

- `:noh`

2. Replacing Text in Vim

Vim provides powerful ways to replace text in your files, using either simple replacements or more complex ones with regular expressions.

Basic Replacement:

- `:s/old/new/`: Replace the first occurrence of old with new on the current line.

- Example: `:s/hello/world/` replaces the first occurrence of "hello" with "world" on the current line.

Global Replacement:

- `:s/old/new/g`: Replace all occurrences of old with new on the current line.

- Example: `:s/hello/world/g` replaces all occurrences of "hello" with "world" on the current line.

Global Replacement Across the Entire File:

- `:%s/old/new/g`: Replace all occurrences of old with new in the entire file.

- Example: `:%s/hello/world/g` replaces every occurrence of "hello" with "world" throughout the entire file.

Replacing with Confirmation:

- `:s/old/new/gc`: Replace all occurrences of old with new, but ask for confirmation before each replacement.

- Vim will ask whether you want to replace each match, with the following options:

- `y`: Yes (replace this occurrence).

- `n`: No (skip this occurrence).

- `a`: All (replace all remaining occurrences without asking).

- `q`: Quit (stop replacing).

- l: Last (replace this one, then stop).

Example of Confirmation

```
:%s/hello/world/gc
```

- This command will search for "hello" and replace it with "world" in the entire file, asking for confirmation before each change.

Replacing in a Specific Range:

You can also specify a range of lines to search and replace in, instead of replacing in the entire file.

- 1,10s/old/new/g: Replace all occurrences of old with new in lines 1 to 10.
- /pattern1/,/pattern2/s/old/new/g: Replace old with new between the lines containing pattern1 and pattern2.

Example:

```
:10,20s/hello/world/g
```

- This command replaces all occurrences of "hello" with "world" between lines 10 and 20.

3. Regular Expressions in Vim Search and Replace

Vim's search and replace commands support regular expressions (regex), making it a powerful tool for complex text manipulation.

Basic Regex in Search and Replace:

- .: Matches any single character (except newline).
- *: Matches zero or more of the preceding character.
- ^: Matches the start of a line.
- \$: Matches the end of a line.
- \b: Word boundary (matches the start or end of a word).
- \d: Matches any digit.
- \w: Matches any word character (letters, digits, or underscores).
- \s: Matches any whitespace character.

Example of Using Regex

- :s/(hello)\(world\)/\2\1/: Swap the words "hello" and "world".
- :s/\d\+/& /: Add a space after every number.

Other Useful Regex Operators

- []: Matches any character in the brackets.
 - Example: :s/[aeiou]/X/g replaces all vowels with "X".
- [^]: Matches any character not in the brackets.
 - Example: :s/[a-zA-Z0-9]//g removes non-alphanumeric characters.
- *: Matches zero or more occurrences of the preceding element.
 - Example: :%s/\w+\/[word]/g replaces every word in the document with [word].

4. Advanced Find and Replace:

Replace Across Multiple Files:

Vim can perform search and replace operations across multiple files using the args list and argdo command.

- :args *.txt: Load all .txt files into the argument list.
- :argdo %s/old/new/g: Replace old with new in all .txt files in the argument list.

Example:

```
:args *.txt
```

```
:argdo %s/old/new/g
```

- This will replace "old" with "new" in all .txt files in the current directory.

Search and Replace Using External Commands:

You can also pipe external tools like grep, sed, or awk into Vim to perform more complex find-and-replace operations. This is useful when you need to use non-Vim tools for advanced regex support or batch processing.

5. Summary of Key Search and Replace Commands

Command Description

/pattern	Search forward for pattern
?pattern	Search backward for pattern
n	Repeat last search in the same direction
N	Repeat last search in the opposite direction
:%s/old/new/g	Replace all occurrences of old with new in the entire file
:%s/old/new/gc	Replace all occurrences with confirmation
:s/old/new/	Replace the first occurrence of old with new on the current line
:s/old/new/g	Replace all occurrences of old with new on the current line
:s/old/new/gc	Replace all occurrences of old with new on the current line with confirmation
:%s/old/new/g	Replace all occurrences of old with new in the entire file
1,10s/old/new/g	Replace from line 1 to line 10
\d, \w, \s	Use regex to find digits, word characters, or whitespace characters

Understanding exec and xargs in Linux

Both `exec` and `xargs` are powerful utilities in Linux that allow you to run commands on multiple files or inputs efficiently. They are often used in combination with tools like `find` or pipelines to execute commands on a list of files or input data.

Let's break down both commands and see how they work with examples.

1. `exec` Command

The `exec` command is often used within `find` to execute commands on files that match certain criteria. It runs a command on each file found by `find` or any other command that produces output. It's extremely useful for running commands on large sets of files or directories.

Syntax:

```
command [arguments] exec command_to_execute {} \;
```

- `{}`: Placeholder for the files that `find` or another command returns.
- `\;`: Delimiters to indicate the end of the command.

The `exec` command runs the specified command on each file that matches the criteria from `find`.

Example 1: Using `exec` with `find`

Let's say you want to find all `.txt` files and then run the `cat` command on each of them to display their content:

```
find . -name "*.txt" -exec cat {} \;
```

- `find . -name "*.txt"`: Find all `.txt` files starting from the current directory.

- `-exec cat {} \;`: For each file found, execute the `cat` command and display its contents.

Example 2: Deleting Files with `exec`

To find all `.log` files and remove them:

```
find . -name "*.log" -exec rm -f {} \;
```

This will search for `.log` files in the current directory and delete them using the `rm` command.

Example 3: Running Multiple Commands with `exec`

You can run multiple commands by using a shell with `-exec`:

```
find . -name "*.log" -exec sh -c 'echo "Deleting {}"; rm -f {}' \;
```

- `sh -c`: Runs a shell command where you can execute multiple commands.
- `echo "Deleting {}"`: Prints a message for each file being deleted.
- `rm -f {}`: Deletes the file.

2. `xargs` Command

`xargs` is another powerful tool in Linux, designed to build and execute command lines from standard input. It is typically used to handle cases where commands like `find` or `grep` return multiple results and we want to pass those results as arguments to another command.

Unlike `exec`, `xargs` takes the output from a command and uses it as input for another command, optimizing performance when working with many arguments.

Syntax:

command | xargs command_to_execute

The main advantage of `xargs` over `exec` is that `xargs` can handle a large number of arguments in a more efficient manner, sometimes batching multiple arguments into a single invocation of the command.

Example 1: Using xargs with find

To find all `.txt` files and count the number of lines in each of them:

```
find . -name "*.txt" | xargs wc -l
```

- `find . -name "*.txt"`: Find all `.txt` files.
- `xargs wc -l`: For each file, run the `wc -l` command to count the lines.

In this example, `xargs` passes the list of `.txt` files to the `wc -l` command, which counts the lines of each file.

Example 2: Deleting Files with xargs

If you want to find and delete all `.log` files, you can use `xargs`:

```
find . -name "*.log" | xargs rm -f
```

- `find . -name "*.log"`: Find all `.log` files.
- `xargs rm -f`: For each file found, run `rm -f` to delete the file.

Example 3: Using xargs to Run Commands with Arguments

You can use `xargs` to pass arguments to a custom command. For example, if you want to compress `.txt` files using `gzip`:

```
find . -name "*.txt" | xargs gzip
```

This will compress all `.txt` files in the current directory using `gzip`.

Example 4: Handling Special Characters with xargs

If your filenames contain spaces or special characters (e.g., newlines), it is a good idea to use the `-0` option in both `find` and `xargs` to ensure correct handling.

- Use `find ... -print0` to print the file names with a null character (`\0`) as the delimiter.
- Use `xargs -0` to tell `xargs` to expect null-terminated input.

For example, to delete files with special characters:

```
find . -name "*.txt" -print0 | xargs -0 rm -f
```

- `-print0`: Tells `find` to print file names with null characters.
- `-0`: Tells `xargs` to expect null-terminated input.

3. Key Differences Between `exec` and `xargs`

Feature	<code>exec</code>	<code>xargs</code>
Input	Takes input directly from <code>find</code> or a pipeline.	Takes input from a pipeline or standard input.
Efficiency	Runs the command once for each file found.	Optimizes by batching multiple arguments and running the command fewer times.
Handling of Input	Each argument (file) is passed individually.	Can handle multiple arguments at once.
Special Characters	Can have trouble with spaces, newlines, or special characters unless handled explicitly.	Handles special characters correctly with <code>-0</code> and <code>-print0</code> options.
Syntax	<code>find ... -exec command {} \;</code>	<code>`command</code>

4. When to Use `exec` vs `xargs`

- Use `exec`:

- When you need to run a command on each file or directory individually, especially for commands that don't take many arguments or for commands that need to run separately for each file.

- Example: When running a command that modifies each file separately or requires independent execution for each file.

- Use xargs:
 - When you have a large number of files and want to optimize the number of times a command is executed.
 - xargs is more efficient because it batches arguments and can reduce the number of command executions.
 - Example: When using commands like rm, cp, mv, or tar to process multiple files at once.

5. Combining find, exec, and xargs

You can combine find, exec, and xargs in powerful ways to manage large sets of files.

Example: Finding and Compressing Files with xargs and exec

You can use xargs for large numbers of files and exec for smaller cases or when a command requires specific handling.

```
find . -name "*.log" -print0 | xargs -0 tar -cvf logs.tar
```

- This combines find with xargs to batch the .log files into a single tar archive. The -print0 and -0 options ensure files with special characters are handled correctly.

6. Conclusion

- exec is best when you need to execute a command on each file found by find, especially for commands that cannot handle multiple arguments at once.
- xargs is best when you want to optimize the execution of a command by passing multiple arguments at once, reducing the number of command invocations.

GREP

The grep command in Linux is one of the most powerful and widely used text-processing tools for searching through files. It allows you to search for patterns within files using regular expressions and return matching lines or content. The name grep stands for Global Regular Expression Print, which refers to its ability to search using regular expressions and print matching lines. Let's dive into the details of grep, its usage, options, and examples.

1. Basic Syntax of grep

```
grep [options] pattern [file...]
```

- pattern: The text or regular expression to search for.
- file: The file or files to search within. If no file is provided, grep reads from standard input.

Basic Example:

To search for the string "error" in a file log.txt:

```
grep "error" log.txt
```

- This will display all lines from log.txt that contain the word "error".

2. Understanding the grep Modes

grep operates primarily in the following modes:

- Basic Search (literal search):
 - By default, grep searches for the exact string or pattern provided (case-sensitive).
- Regular Expressions:
 - grep supports basic regular expressions (BRE) by default, but you can use extended regular expressions (ERE) by passing the -E option (or using egrep, which is the same as grep -E).
- Case Sensitivity:
 - By default, grep is case-sensitive, but you can make it case-insensitive with the -i option.

3. Commonly Used Options in grep

Here are the most frequently used options with grep:

Option	Description	Example
-i	Ignore case while searching.	grep -i "error" log.txt

-v Invert match; display lines that do not match the pattern. `grep -v "error" log.txt`

-r / -R Search recursively through directories. `grep -r "error" /var/log/`

-l Show only the names of files that contain the pattern. `grep -l "error" *.log`

-L Show only the names of files that do not contain the pattern. `grep -L "error" *.log`

-n Display line numbers along with matching lines. `grep -n "error" log.txt`

-c Count the number of matching lines. `grep -c "error" log.txt`

-H Print the filename in the output (useful when searching multiple files). `grep -H "error" *.log`

-h Suppress printing of filenames (default behavior when only one file is searched). `grep -h "error" *.log`

-o Print only the matched portion of the line, not the entire line. `grep -o "error" log.txt`

-w Match only whole words (pattern must not be part of a larger word). `grep -w "error" log.txt`

-x Match only whole lines (entire line must match the pattern). `grep -x "error" log.txt`

-A NUM Print NUM lines of context after the matching line. `grep -A 2 "error" log.txt`

-B NUM Print NUM lines of context before the matching line. `grep -B 2 "error" log.txt`

-C NUM Print NUM lines of context before and after the matching line. `grep -C 2 "error" log.txt`

-e Use multiple patterns; allows you to specify multiple patterns. `grep -e "error" -e "warning" log.txt`

-f FILE Read patterns from a file. This is useful when you want to search for many patterns stored in a file. `grep -f patterns.txt log.txt`

--color=auto Highlight matching strings in color (useful for visibility). `grep --color=auto "error" log.txt`

4. Searching with Regular Expressions (Regex)

grep supports basic regular expressions (BRE) by default, but with the -E option, you can use extended regular expressions (ERE). Here's how regular expressions work in grep:

Basic Regular Expressions (BRE)

Symbol	Description	Example
.	Matches any single character.	<code>grep "a.b" file.txt</code> (matches "acb", "a1b", etc.)
^	Matches the beginning of a line.	<code>grep "^Start" file.txt</code> (matches lines starting with "Start")
\$	Matches the end of a line.	<code>grep "end\$" file.txt</code> (matches lines ending with "end")
*	Matches zero or more of the preceding character.	<code>grep "a*" file.txt</code> (matches lines with zero or more "a"s)
[]	Matches any one of the characters inside the brackets.	<code>grep "[aeiou]" file.txt</code> (matches lines containing any vowel)
[^]	Matches any character that is not inside the brackets.	<code>grep "[^aeiou]" file.txt</code> (matches lines without vowels)
\	Escape special characters.	<code>grep "a*b" file.txt</code> (matches "a*b" literally, not as a pattern)

Extended Regular Expressions (ERE)

When using the -E option, grep supports extended features like +, ?, {}, |, etc.

Symbol	Description	Example
+	Matches one or more of the preceding character.	<code>grep -E "a+b" file.txt</code> (matches "ab", "aab", "aaab", etc.)
?	Matches zero or one of the preceding character.	<code>grep -E "a?b" file.txt</code> (matches "b" and "ab")
{n,m}	Matches between n and m occurrences of the preceding character.	<code>grep -E "a{2,4}" file.txt</code> (matches "aa", "aaa", and "aaaa")
	Acts as OR, matching either the pattern before or after it.	

Example of Regular Expressions in grep

- Find all lines that start with "Hello":
grep "^Hello" file.txt
- Find all lines that contain either "error" or "warning":
grep -E "error|warning" file.txt
- Find lines that contain "abc", followed by any character, and then "xyz":
grep "abc.xyz" file.txt

5. Practical Examples

Here are a few practical use cases of grep:

Example 1: Search for a Pattern in Multiple Files

To search for the string "error" in all .log files in the current directory:

```
grep "error" *.log
```

Example 2: Case-Insensitive Search

To search for "error" regardless of case:

```
grep -i "error" log.txt
```

Example 3: Search Recursively in Directories

To search for "error" in all files within the current directory and subdirectories:

```
grep -r "error" .
```

Example 4: Count the Occurrences

To count how many times the word "error" appears in log.txt:

```
grep -c "error" log.txt
```

Example 5: Show Only Matching Part of the Line

To show only the matching word "error" in the file:

```
grep -o "error" log.txt
```

Example 6: Show Line Numbers

To show the line numbers along with the matching lines:

```
grep -n "error" log.txt
```

Example 7: Display Lines Before/After Match

To show 3 lines before and after the match:

```
grep -C 3 "error" log.txt
```

Example 8: Search for Whole Words

To search for the whole word "error" (not part of another word like "terror"):

```
grep -w "error" log.txt
```

Example 9: Using grep with ps to Find Processes

To find processes related to "nginx":

```
ps aux | grep "nginx"
```

6. Conclusion

grep is an indispensable tool for anyone working with text files in Linux. It is efficient, versatile, and powerful, supporting everything from simple literal searches to complex regular expressions. Whether you're analyzing logs, searching through code, or filtering command output, grep is your go-to tool for finding patterns quickly.

Mastering grep will significantly enhance your ability to search and process text in Linux, making it a vital skill for system administrators, developers, and data analysts.

FIND

The find command in Linux is one of the most powerful and flexible tools used for searching files and directories in a filesystem based on various criteria, such as name, size, modification time, type, permissions, etc. It also allows you to execute actions on the files it finds.

Let's dive into the details of how the find command works, its options, and practical use cases.

1. Basic Syntax of the find Command

```
find [path] [expression]
```

- path: The starting directory where the search will begin. If no path is specified, find will search from the current directory (.).

- expression: The search criteria (such as name, type, size, etc.) and actions to apply on the files that match.

Example:

```
find /home/user -name "*.txt"
```

This command will search for all files with the .txt extension under /home/user directory and its subdirectories.

2. Commonly Used find Options

Path:

- .: Current directory.
- /path/to/dir: The directory you want to search within.

Search Criteria:

- -name pattern: Search for files that match the pattern (case-sensitive).
- find . -name "*.txt"
- -iname pattern: Search for files that match the pattern (case-insensitive).

- find . -iname "*.txt"
- -type [f|d|l]: Search for specific types of files:
- f: Regular files.
- d: Directories.
- l: Symbolic links.
- find . -type f -name "*.txt" # Search for regular files

```
find . -type d -name "backup" # Search for directories named "backup"
```

- -size [+|-]n: Search for files of a certain size.
- +n: Greater than n (e.g., +1M for greater than 1MB).
- -n: Less than n (e.g., -10k for files smaller than 10KB).
- n: Exactly n (e.g., 5M for exactly 5MB).
- find . -size +100M # Files larger than 100MB

```
find . -size -10k # Files smaller than 10KB
```

- -mtime [+|-]n: Search for files modified n days ago.
- +n: Modified more than n days ago.
- -n: Modified less than n days ago.
- n: Modified exactly n days ago.
- find . -mtime +7 # Files modified more than 7 days ago

```
find . -mtime -1 # Files modified in the last day
```

- -atime [+|-]n: Search for files accessed n days ago (similar to -mtime).
- find . -atime +30 # Files accessed more than 30 days ago
- -ctime [+|-]n: Search for files changed (status change) n days ago.
- find . -ctime -7 # Files whose metadata (permissions, owner, etc.) changed in the last 7 days

- -user username: Search for files owned by the specified user.
- find . -user john
- -group groupname: Search for files belonging to a specific group.
- find . -group admins
- -perm mode: Search for files with specific permissions.
- mode can be expressed in numeric (octal) or symbolic form.
- find . -perm 755 # Files with 755 permissions

```
find . -perm -u+x # Files where the user has execute permission
```

3. Executing Commands on Files Found

The -exec option allows you to run commands on the files found by find. This is one of the most powerful features of find.

Basic Syntax of -exec

```
find [path] [expression] -exec [command] {} \;
```

- {}: Placeholder for the file or directory found by find.
- \;; End of the exec command.

Examples:

- Delete all .log files:
- find . -name "*.log" -exec rm -f {} \;
- Change the permissions of all .sh files to 755:

- `find . -name "*.sh" -exec chmod 755 {} \;`
- Count lines of all .txt files:
- `find . -name "*.txt" -exec wc -l {} \;`
- Using -exec with sh for multiple commands:

You can use sh -c to execute multiple commands for each file.

- `find . -name "*.txt" -exec sh -c 'echo "Processing {}"; wc -l {}' \;`

Using + Instead of \

By default, -exec runs the specified command for each file individually. If you use + instead of \;, find will pass all the matching files to the command at once, which can be much more efficient when dealing with a large number of files.

- Remove all .log files using + (faster for many files):
- `find . -name "*.log" -exec rm -f {} +`

4. Search with Multiple Conditions

You can combine different conditions and actions using logical operators.

Logical AND (-a)

By default, conditions are combined with logical AND, meaning all conditions must be true for a file to be selected.

- Find .txt files that are larger than 1MB:
- `find . -name "*.txt" -size +1M`

Logical OR (-o)

You can use -o (OR) to combine different conditions where only one condition needs to be true for the file to be selected.

- Find .txt or .log files:
- `find . \( -name "*.txt" -o -name "*.log" \)`

◦ The parentheses () are used to group conditions. You need to escape them with backslashes \(and \)).

Negation (! or -not)

Use ! or -not to negate a condition, meaning files that do not match the condition.

- Find files that are not .txt:
- `find . -not -name "*.txt"`

5. Search for Empty Files and Directories

- Find empty files:
- `find . -type f -empty`
- Find empty directories:
- `find . -type d -empty`

6. Other Useful Examples

- Find files modified in the last 24 hours:
- `find . -mtime -1`
- Find files larger than 100MB:
- `find . -size +100M`
- Find files and list their details using ls:
- `find . -type f -exec ls -l {} \;`
- Find and change ownership of all .txt files to user alice:
- `find . -name "*.txt" -exec chown alice {} \;`
- Find all files with a certain extension and compress them with gzip:
- `find . -name "*.log" -exec gzip {} \;`

7. Combining find with Other Commands

- Using find with xargs:

xargs is often used with find to optimize the execution of commands by passing the results as arguments to a command.

- Find and remove .log files:
- `find . -name "*.log" | xargs rm -f`
- Find and zip all .txt files:
- `find . -name "*.txt" | xargs tar -czf archive.tar.gz`

8. Summary of find Command Syntax

Option/Expression	Description	Example
-name pattern	Search for files by name	<code>find . -name "*.txt"</code>
-type f	Search for files (not directories)	<code>find . -type f -name "*.sh"</code>

-size	Search based on file size	find . -size +10M
-mtime	Search based on modification time	find . -mtime -7
-exec	Execute a command on files found	find . -name "*.log" -exec rm -f {} \;
-print	Print the file path (default behavior)	find . -name "*.txt" -print
-user	Search for files owned by a specific user	find . -user john
-group	Search for files belonging to a specific group	find . -group admins
-perm	Search based on file permissions	find . -perm 755

9. Conclusion

The find command is extremely versatile and a core part of any Linux user's toolkit. It's useful for searching for files based on numerous criteria and performing actions on them. Whether you're looking for files of a certain type, size, or age, or need to execute commands on the files you find, find can do it all.

CUT

The cut command in Linux is used for cutting sections from each line of a file or stream and outputting those sections. It is often used for extracting specific columns or fields from text files, making it a valuable tool for text processing and data extraction. You can specify the delimiter (separator) that divides the fields, and then choose which columns or fields to extract.

1. Basic Syntax of cut

```
cut [options] [file...]
```

Where the options allow you to specify how to select the columns, fields, or characters to cut from the input.

Example:

```
cut -d ',' -f 1,3 input.txt
```

This command will cut out the first and third fields (separated by commas) from the file input.txt.

2. Commonly Used Options with cut

Option	Description	Example
--------	-------------	---------

-d	Set the delimiter character. By default, cut assumes tab-separated fields.	cut -d ',' -f 1,3 file.csv (using comma as delimiter)
----	--	---

-f	Specify the field(s) to be cut. Fields are 1-based. Can be a single field or a range of fields.	cut -f 1,2 file.txt (selects the first and second fields)
----	---	---

-b	Select specific bytes from each line (1-based).	cut -b 1-5 file.txt (selects the first 5 bytes)
----	---	---

-c	Select specific characters from each line (1-based).	cut -c 1-5 file.txt (selects the first 5 characters)
----	--	--

--complement	Complement the selection; i.e., exclude the selected fields/bytes/characters.	cut -d ',' -f 1,2 --complement file.csv (exclude 1st and 2nd fields)
--------------	---	--

--output-delimiter	Specify a delimiter for the output (different from the input delimiter).	cut -d ',' -f 1,2 --output-delimiter=":" file.csv
--------------------	--	---

3. Cutting Fields Using -f Option

The -f option is used to extract specific fields or columns from the input.

Example 1: Cutting a Single Field

```
cut -d ',' -f 2 data.csv
```

- This will output the second column of the file data.csv, assuming the columns are separated by commas.

Example 2: Cutting Multiple Fields

```
cut -d ',' -f 1,3,5 data.csv
```


- This will output the 1st, 3rd, and 5th columns from data.csv.

Example 3: Cutting a Range of Fields

```
cut -d ',' -f 2-4 data.csv
```

- This will output fields 2 through 4 from each line in the file.

Example 4: Cutting All Fields Except the First

You can use the `--complement` option to exclude specific fields. To exclude the first field and display all others:

```
cut -d ',' -f 1 --complement data.csv
```

4. Cutting Characters Using -c Option

The `-c` option allows you to select specific characters from each line.

Example 1: Cutting a Range of Characters

```
cut -c 1-5 data.txt
```

- This will output the first 5 characters from each line of the file.

Example 2: Cutting Multiple Character Ranges

```
cut -c 1-3,7-9 data.txt
```

- This will output characters 1 to 3 and 7 to 9 from each line.

5. Cutting Bytes Using -b Option

The `-b` option works similarly to `-c`, but it operates at the byte level. This is useful when working with binary files or files that contain multi-byte characters (like UTF-8).

Example 1: Cutting a Range of Bytes

```
cut -b 1-5 data.bin
```

- This will output the first 5 bytes from each line (or data chunk) in data.bin.

Example 2: Cutting Multiple Byte Ranges

```
cut -b 1-3,7-10 data.bin
```

- This will output bytes 1 to 3 and 7 to 10 from each line of the binary file.

6. Specifying Delimiters

By default, `cut` assumes that fields are separated by a tab character (`\t`). You can specify a custom delimiter using the `-d` option.

Example: Using a Comma as a Delimiter

If you have a CSV file and want to extract the first and third columns:

```
cut -d ',' -f 1,3 file.csv
```

- This will treat commas as field separators and extract the first and third columns.

Example: Using a Space as a Delimiter

```
cut -d ' ' -f 1,2 file.txt
```

- This treats spaces as delimiters and extracts the first and second columns.

7. Handling Output Delimiters with --output-delimiter

You can change the output delimiter using the `--output-delimiter` option. This is useful when you want to output the extracted fields with a custom delimiter.

Example: Changing the Output Delimiter to Colon

```
cut -d ',' -f 1,2 --output-delimiter=":" file.csv
```

- This will output the first and second fields, but separate them with a colon (`:`) instead of the default tab.

8. Practical Use Cases of cut

Example 1: Extracting IP Addresses from a Log File

If you have a log file with IP addresses in the first field (separated by spaces):

```
cut -d ' ' -f 1 access.log
```

- This will extract and print all IP addresses from the first field.

Example 2: Extracting Usernames from /etc/passwd

In the `/etc/passwd` file, usernames are the first field, separated by colons:

```
cut -d ':' -f 1 /etc/passwd
```

- This will print the usernames from the `/etc/passwd` file.

Example 3: Extracting Date from Log Entries

If you have a log file with the date in the first 10 characters of each line:

```
cut -c 1-10 log.txt
```

- This will extract and print the date (assuming it's at the start of each line).

Example 4: Extracting Domain Names from Email Addresses

If you have a list of email addresses and want to extract the domain name (part after @):

```
cut -d '@' -f 2 emails.txt
```

- This will print the domain names (e.g., gmail.com, yahoo.com) from the email addresses.

9. Combining cut with Other Commands

You can combine cut with other commands using pipes (|) for more advanced processing.

Example: Using cut with sort to Sort Unique Entries

To extract the first field from a CSV file and sort the unique entries:

```
cut -d ',' -f 1 data.csv | sort | uniq
```

Example: Using cut with grep

You can use cut in combination with grep to filter and cut out specific fields from matching lines:

```
grep "error" log.txt | cut -d ' ' -f 1,2
```

- This filters lines containing the word "error" and then cuts out the first and second fields.

10. Conclusion

The cut command is a simple yet powerful tool for text processing in Linux. It allows you to extract specific fields, columns, or characters from a file or output stream, making it ideal for tasks such as parsing CSV files, log files, or extracting specific data from structured text.

By combining cut with other Linux utilities (like grep, sort, and awk), you can perform complex text manipulation efficiently on the command line.

SED

The sed (stream editor) command in Linux is a powerful tool for performing basic text transformations on an input stream (a file or input from a pipeline). sed is primarily used for text substitution, deletion, insertion, and other modifications in files or streams.

Let's dive into the details of the sed command, its syntax, options, and examples.

1. Basic Syntax of sed

```
sed [options] 'command' file
```

- command: The operation you want to perform (e.g., substitution, deletion, etc.).

- file: The file on which the operation will be applied (optional; if not provided, sed reads from stdin).

2. Commonly Used sed Commands

Substitution (Search and Replace)

The most common use of sed is to substitute a pattern in the text.

```
sed 's/pattern/replacement/' file
```

- s/pattern/replacement/: This is the substitution command, where:
 - pattern is the text or regular expression to search for.
 - replacement is the text to replace the pattern.

Example 1: Replace the first occurrence of "apple" with "orange" in each line:

```
sed 's/apple/orange/' file.txt
```

Example 2: Replace all occurrences of "apple" with "orange" in each line (add g at the end for global replacement):

```
sed 's/apple/orange/g' file.txt
```

Substitution with Case Sensitivity

By default, sed is case-sensitive. If you want a case-insensitive substitution, use the I flag (GNU sed only).

```
sed 's/apple/orange/I' file.txt
```

Replacing Only the Nth Occurrence

To replace only the Nth occurrence of a pattern on each line:

```
sed 's/apple/orange/3' file.txt
```

- This will replace only the third occurrence of "apple" with "orange" on each line.

3. Deleting Lines

Delete Specific Line(s)

- Delete a specific line by line number:
 - `sed '3d' file.txt`
 - This deletes line 3 from the file.
- Delete a range of lines:
 - `sed '3,5d' file.txt`
 - This deletes lines 3 to 5.
- Delete lines matching a pattern:
 - `sed '/pattern/d' file.txt`
 - This deletes all lines containing "pattern".

4. Inserting and Appending Text

Insert Text Before a Line

To insert text before a specific line:

```
sed '3i\This is the inserted text' file.txt
```

- This will insert "This is the inserted text" before line 3.

Append Text After a Line

To append text after a specific line:

```
sed '3a\This is the appended text' file.txt
```

- This will append "This is the appended text" after line 3.

5. Modifying the File In-Place

By default, sed outputs the modified content to the terminal. If you want to modify the file in place, use the `-i` option. You can optionally provide a suffix for creating a backup file.

Modify the file directly without backup:

```
sed -i 's/apple/orange/g' file.txt
```

- This will replace all occurrences of "apple" with "orange" in the file `file.txt`.

Modify the file and create a backup:

```
sed -i.bak 's/apple/orange/g' file.txt
```

- This creates a backup of the original file as `file.txt.bak` and performs the substitution on `file.txt`.

6. Using Regular Expressions in sed

sed supports regular expressions, which makes it a powerful tool for complex text manipulations.

Example 1: Match any character (.)

```
sed 's/a.b/xyz/' file.txt
```

- This will match any character between a and b (e.g., "a1b", "acb", etc.) and replace it with xyz.

Example 2: Match the beginning of a line (^)

```
sed 's/^Hello/Hi/' file.txt
```

- This will replace Hello with Hi at the beginning of each line.

Example 3: Match the end of a line (\$)

```
sed 's/world$/planet/' file.txt
```

- This will replace world with planet at the end of each line.

Example 4: Use parentheses for grouping and \1 for backreference

```
sed 's/\(apple\) and \(orange\)/\2 and \1/' file.txt
```

- This swaps "apple" and "orange" if they appear in the same line, using backreferences (\1, \2).

7. sed with Pipes and Other Commands

You can use sed with other commands through pipes for more complex workflows.

Example: Using sed with grep

You can filter lines with grep and then modify them with sed.

```
grep "apple" file.txt | sed 's/apple/orange/g'
```

- This will find all lines containing "apple" and replace "apple" with "orange".

Example: Using sed to Change the Filename with ls

You can list files and use sed to change part of their names:

```
ls | sed 's/.txt/.bak/'
```

- This will change the .txt extension to .bak for each file in the directory.

8. Using sed for Advanced Text Manipulations

Replace Only the First Occurrence in Each Line

By default, sed will replace all occurrences of the pattern in a line. To replace only the first occurrence, simply omit the g flag:

```
sed 's/apple/orange/' file.txt
```

- This will replace only the first occurrence of "apple" with "orange" on each line.

Substitute a Pattern Only on Certain Lines

You can limit substitutions to specific lines:

- Replace "apple" with "orange" on line 3 only:
- sed '3s/apple/orange/' file.txt

Using sed with Multiple Commands

You can perform multiple operations on a file using the -e option or by chaining commands with ;.

- Using -e to apply multiple commands:
- sed -e 's/apple/orange/' -e 's/banana/pear/' file.txt
- Chaining commands with ;:
- sed 's/apple/orange/; s/banana/pear/' file.txt

9. Special sed Commands

- -n option (Suppress output): When you use the -n option, sed will suppress automatic output and only print lines where explicitly told to do so with the p command.

- Example: Print only lines that contain "apple":

- sed -n '/apple/p' file.txt

- p command: This command explicitly tells sed to print a line. It's useful in conjunction with the -n option.

- Example: Print lines 2 to 4:

- sed -n '2,4p' file.txt

- q command (Quit after N lines): The q command can be used to exit after processing a certain number of lines.

- Example: Exit after line 5:

- sed '5q' file.txt

10. Summary of sed Commands

Command	Description	Example
s/pattern/replacement/	Substitution (search and replace)	sed 's/apple/orange/g' file.txt
d	Delete a line	sed '2d' file.txt
i	Insert text before a line	sed '2i\New text' file.txt
a	Append text after a line	sed '2a\New text' file.txt
-i	Modify file in-place	sed -i 's/apple/orange/g' file.txt
-n	Suppress automatic output	sed -n '3p' file.txt
p	Print matching lines	sed -n '/apple/p' file.txt
q	Quit after processing N lines	sed '5q' file.txt
-e	Specify multiple commands	sed -e 's/apple/orange/' -e 's/banana/pear/' file.txt

11. Conclusion

The sed command is an essential tool for stream editing and performing complex text transformations in Unix-like systems. Its power lies in the ability to modify text in files or streams efficiently with regular expressions, substitutions, deletions, insertions, and more.

Mastering sed will greatly enhance your ability to automate text processing, whether you're working with log files, configuration files, or data extraction tasks

SORT

The sort command in Linux is used to arrange lines of text in a specified order. It is highly versatile and can sort data in ascending or descending order, handle numeric sorting, and work with custom delimiters or specific fields. Sorting can be done alphabetically, numerically, or by other criteria such as date or size.

Let's break down the sort command, its common options, and practical examples.

1. Basic Syntax of sort

`sort [options] [file...]`

- **file:** The file or files to be sorted. If no file is specified, sort reads from stdin.

- **options:** Flags that control how the sorting is done.

2. Commonly Used sort Options

Option	Description	Example
-n	Sort numerically (instead of alphabetically).	<code>sort -n file.txt</code>
-r	Reverse the order (sort in descending order).	<code>sort -r file.txt</code>
-k	Sort by a specific column or field.	<code>sort -k 2 file.txt</code>
-t	Specify a delimiter (default is whitespace).	<code>sort -t ',' -k 2 file.csv</code>
-u	Only output unique lines (removes duplicates).	<code>sort -u file.txt</code>
-f	Ignore case when sorting.	<code>sort -f file.txt</code>
-b	Ignore leading blanks or spaces.	<code>sort -b file.txt</code>
-o	Write output to a file instead of standard output.	<code>sort file.txt -o sorted_file.txt</code>
-M	Sort by month name (e.g., Jan, Feb, etc.).	<code>sort -M file.txt</code>
-g	Sort by general numerical value (floating point).	<code>sort -g file.txt</code>
-c	Check whether the input is sorted.	<code>sort -c file.txt</code>

3. Basic Sorting Examples

Example 1: Alphabetical Sorting (Ascending)

Sort the lines of a file alphabetically (default behavior):

`sort file.txt`

- This will output the lines of file.txt sorted alphabetically in ascending order.

Example 2: Reverse Alphabetical Sorting

Sort the lines of a file in reverse (descending) order:

`sort -r file.txt`

Example 3: Numeric Sorting

If you have a file with numbers and want to sort them in numeric order:

`sort -n file.txt`

- This will sort lines in ascending order based on numeric values.

Example 4: Sort in Reverse Numeric Order

To sort numbers in descending order:

`sort -nr file.txt`

4. Sorting by Specific Fields/Columns

Example 1: Sort by the Second Column (Field)

To sort by the second column, use the -k option. By default, sort uses whitespace as a delimiter between fields.

`sort -k 2 file.txt`

- This will sort the lines based on the second column.

Example 2: Sort by a Specific Delimited Field

If you have a CSV file and want to sort by a specific column (e.g., the second column in a comma-separated file), use the `-t` option to specify the delimiter:
`sort -t ',' -k 2 file.csv`

- This will sort the lines based on the second column, treating commas as the delimiter.

Example 3: Sort by Multiple Fields

You can specify multiple fields to sort by. For example, to first sort by the second field, and then by the third field:

`sort -k 2,2 -k 3,3 file.txt`

- This will first sort by the second column and, if there are ties, it will then sort by the third column.

5. Handling Case Sensitivity

Example 1: Case-Sensitive Sorting

By default, sort is case-sensitive, meaning uppercase letters are sorted before lowercase ones.

`sort file.txt`

Example 2: Case-Insensitive Sorting

To ignore case while sorting:

`sort -f file.txt`

- This sorts lines without considering the case of the letters.

6. Handling Duplicates

Example 1: Sort and Remove Duplicates

If you want to sort and remove any duplicate lines:

`sort -u file.txt`

- This will sort the file and output only unique lines.

Example 2: Count Unique Lines

You can combine sort with `uniq` to count the number of unique occurrences of each line.

`sort file.txt | uniq -c`

- This will show how many times each line occurs in the file.

7. Sorting Based on Different Criteria

Example 1: Sort by Month Name

If you have a file with month names (e.g., Jan, Feb, Mar) and you want to sort by month order, use the `-M` option:

`sort -M file.txt`

- This will sort the months in chronological order (January, February, March, etc.).

Example 2: Sort by General Numeric Value (Floating Point)

Use the `-g` option for general numeric sorting, which works for both integer and floating-point values:

`sort -g file.txt`

Example 3: Check If File is Already Sorted

To check if a file is sorted, use the `-c` option:

`sort -c file.txt`

- If the file is already sorted, sort will not produce any output. If not, it will indicate where the sorting issue occurs.

8. Writing the Sorted Output to a File

You can use the `-o` option to write the sorted data directly to a file.

`sort file.txt -o sorted_file.txt`

- This sorts `file.txt` and saves the result in `sorted_file.txt`.

9. Sorting with Pipes

You can combine sort with other commands using pipes to sort the output of other commands.

Example 1: Sort the Output of ps Command

```
ps aux | sort -k 3 -n
```

- This will sort the output of the ps aux command by the third column (CPU usage) in numeric order.

Example 2: Sort by File Size

To sort a list of files by size, use ls and pipe it to sort:

```
ls -l | sort -k 5 -n
```

- This sorts the files by size (the 5th column) in ascending order.

10. Sorting in Reverse with Custom Delimiters

Example 1: Reverse Sorting with Custom Delimiter

To reverse the order of lines sorted by a custom delimiter, use -t for the delimiter and -r for reverse order.

```
sort -t ',' -k 2 -r file.csv
```

- This sorts a CSV file by the second column in reverse order.

11. Combining sort with Other Tools

You can combine sort with other commands like awk, cut, uniq, and grep for more complex processing.

Example: Sorting and Filtering with grep

Sort the lines containing "apple" and remove duplicates:

```
grep "apple" file.txt | sort -u
```

Example: Sorting and Extracting Specific Fields

If you want to sort and extract specific columns, you can use cut or awk before sorting:

```
awk '{print $2, $1}' file.txt | sort
```

- This swaps the first and second columns and sorts the lines.

12. Conclusion

The sort command is an essential tool for sorting text data in Linux. Whether you are working with numbers, dates, or simple text, sort offers a range of options to tailor the sorting to your needs. By combining sort with other commands and options, you can perform complex text processing and data manipulation tasks efficiently.

UNIQ

The uniq command in Linux is used to filter out duplicate lines in a file or input stream. It's typically used in conjunction with the sort command because uniq only removes adjacent duplicate lines, so sorting the input beforehand ensures that all duplicates are next to each other.

Here's a detailed breakdown of the uniq command, its common options, and practical examples.

1. Basic Syntax of uniq

```
uniq [options] [input_file] [output_file]
```

- input_file: The file or input stream you want to process.
- output_file (optional): If you want to redirect the output to a file.

By default, uniq filters out adjacent duplicate lines.

2. Commonly Used uniq Options

Option	Description	Example
-u	Print only unique lines (exclude duplicates).	uniq -u file.txt
-d	Print only duplicate lines.	uniq -d file.txt
-c	Prefix lines with the number of occurrences.	uniq -c file.txt
-i	Ignore case when comparing lines.	uniq -i file.txt

-f N Skip the first N fields/columns before comparing. `uniq -f 1 file.txt`
-s N Skip the first N characters before comparing. `uniq -s 3 file.txt`
-w N Compare the first N characters of each line. `uniq -w 4 file.txt`
-o FILE Redirect output to a file. `uniq -c file.txt > output.txt`

3. Basic Examples of `uniq`

Example 1: Remove Duplicates in a File

To remove adjacent duplicate lines in a file:

`uniq file.txt`

- This will display the lines in `file.txt` without any consecutive duplicates.

Example 2: Count Occurrences of Each Line

To count how many times each line appears, prefix each line with the count of its occurrences:

`uniq -c file.txt`

- This will output the lines in `file.txt`, with the count of how many times each line appears, followed by the line itself.

Example 3: Display Only Duplicate Lines

To display only the lines that appear more than once:

`uniq -d file.txt`

- This will display only the lines that appear more than once in the input.

Example 4: Display Only Unique Lines (Excluding Duplicates)

To display only lines that appear once, excluding duplicates:

`uniq -u file.txt`

- This will display only the lines that appear exactly once in `file.txt`.

4. Working with Sorted Data

`uniq` removes adjacent duplicates, so it's common to sort the data before using `uniq` to ensure that all duplicates are adjacent.

Example 1: Sort and Remove Duplicates

If your file has unsorted data and you want to remove all duplicates:

`sort file.txt | uniq`

- This will first sort the file, then remove duplicates.

Example 2: Sort and Count Occurrences

To sort the lines first, then count how many times each line appears:

`sort file.txt | uniq -c`

- This will sort the lines, count occurrences, and display the results.

Example 3: Sort and Display Only Duplicates

To sort the file and display only duplicate lines (those that appear more than once):

`sort file.txt | uniq -d`

5. Ignoring Case with `uniq`

By default, `uniq` is case-sensitive, meaning "apple" and "Apple" will be treated as different lines. You can use the `-i` option to ignore case differences.

Example: Case-Insensitive Duplicate Removal

`uniq -i file.txt`

- This will remove duplicates, ignoring case differences (i.e., "apple" and "Apple" will be treated as the same).

6. Skipping Fields and Characters

You can control which parts of the line `uniq` should consider for comparison by skipping a specific number of fields or characters.

Example 1: Skip First N Fields

To skip the first N fields (columns), use the -f option. For example, if you want to skip the first column (assuming the fields are space-separated):

```
uniq -f 1 file.txt
```

- This skips the first field and compares the remaining parts of each line.

Example 2: Skip the First N Characters

To skip the first N characters of each line, use the -s option:

```
uniq -s 3 file.txt
```

- This will compare lines starting from the 4th character.

Example 3: Compare First N Characters

To compare only the first N characters of each line, use the -w option:

```
uniq -w 5 file.txt
```

- This will compare the first 5 characters of each line to determine duplicates.

7. Writing Output to a File

You can redirect the output of uniq to a file using redirection or the -o option.

Example 1: Redirect Output to a File

```
uniq file.txt > unique_lines.txt
```

- This will store the unique lines in unique_lines.txt.

Example 2: Redirect Counted Output to a File

To save the count of occurrences to a file:

```
uniq -c file.txt > counted_file.txt
```

8. Practical Use Cases of uniq

Example 1: Count Occurrences of Words in a List

If you have a list of words or phrases, you can use sort and uniq to count occurrences:

```
sort words.txt | uniq -c
```

- This will count the occurrences of each unique word in words.txt.

Example 2: Filter Out Consecutive Duplicates in Log Files

If you're working with log files and want to remove consecutive duplicate log entries (e.g., repetitive warnings):

```
uniq logs.txt
```

Example 3: Extracting Unique Email Domains

Given a list of email addresses, extract the unique email domains:

```
cut -d '@' -f 2 emails.txt | sort | uniq
```

- This will cut out the domain part of each email, sort them, and then display only unique domains.

9. Combining uniq with Other Commands

You can combine uniq with other commands using pipes to enhance its functionality.

Example 1: Combine grep, sort, and uniq

If you only want to find and list unique lines that contain a specific word, you can combine grep, sort, and uniq:

```
grep "error" log.txt | sort | uniq
```

Example 2: Using uniq with awk to Extract and Count Fields

Use awk to extract specific fields from a file and then use uniq to count them:

```
awk '{print $2}' file.txt | sort | uniq -c
```

- This will print the second column from file.txt, sort the output, and then display the count of each unique value in that column.

10. Conclusion

The `uniq` command is a simple but essential tool for working with text data in Linux. It is used to remove duplicate lines, count occurrences, and handle case insensitivity, among other things. When combined with `sort`, `grep`, and other tools, `uniq` can be extremely powerful for processing large datasets and log files.

AWK

The `awk` command is one of the most powerful text-processing tools available in Linux. It is a versatile programming language designed for pattern scanning and processing. It operates on a line-by-line basis, reading input line by line and performing specified operations (like searching, transforming, or printing) on each line.

Here's a detailed explanation of the `awk` command, its syntax, common options, and practical examples.

1. Basic Syntax of awk

`awk 'pattern { action }' input_file`

- `pattern`: Specifies the condition for selecting lines. If the pattern is true for a line, the action is executed.
- `action`: Specifies what to do when the pattern matches the line (e.g., `print` specific fields, modify the data, etc.).
- `input_file`: The file on which `awk` will operate. If no file is provided, `awk` reads from standard input (`stdin`).

The basic structure of an `awk` command is:

`awk 'pattern { action }' file`

If no pattern is provided, `awk` assumes the action should be applied to all lines. If no action is provided, the default action is to print the line.

2. Basic Examples of awk

Example 1: Print All Lines of a File

If you simply want to print all lines from a file, you can omit both the pattern and the action (the default action is `print`):

`awk '{ print }' file.txt`

- This prints every line of the file `file.txt`.

Example 2: Print Specific Fields

By default, `awk` splits each line into fields, with the default delimiter being whitespace (spaces or tabs). The fields are numbered as `$1`, `$2`, `$3`, etc. `$0` represents the entire line.

`awk '{ print $1, $3 }' file.txt`

- This will print the first and third fields of each line from the file `file.txt`.

Example 3: Print the First Field of Each Line

`awk '{ print $1 }' file.txt`

- This will print only the first field of each line from the file.

3. Using Patterns with awk

You can specify patterns to select lines that match a specific condition.

Example 1: Print Lines That Match a Pattern

To print lines that contain the word "apple":

`awk '/apple/ { print }' file.txt`

- This will print all lines from `file.txt` that contain the word "apple".

Example 2: Print Lines Matching Multiple Conditions

You can combine multiple conditions using logical operators like `&&` (AND) and `||` (OR).

`awk '$1 == "apple" && $3 > 10 { print $1, $3 }' file.txt`

- This will print the first and third fields of lines where the first field is "apple" and the third field is greater than 10.

Example 3: Print Lines Based on Line Number

You can also use patterns based on the line number. For instance, to print the 2nd line:

```
awk 'NR == 2 { print }' file.txt
```

- This prints only the second line of the file.
- NR: Built-in variable that represents the line number.

4. Built-In Variables in awk

awk provides several built-in variables that can be used in patterns or actions:

- \$0: The entire current line.
- \$1, \$2, ...: The individual fields in the current line (fields are space/tab-separated by default).
- NF: The number of fields in the current line.
- NR: The number of records (lines) processed so far.
- FS: The field separator (default is whitespace).
- OFS: The output field separator (default is space).
- ORS: The output record separator (default is a newline).
- RS: The input record separator (default is newline).

Example 1: Print the Number of Fields in Each Line

```
awk '{ print NF }' file.txt
```

- This will print the number of fields in each line of the file.

Example 2: Print Line Number and Line Content

```
awk '{ print NR, $0 }' file.txt
```

- This will print the line number followed by the line content.

Example 3: Change the Field Separator

To change the field separator (e.g., to comma), set the FS variable:

```
awk 'BEGIN { FS="," } { print $1, $2 }' file.csv
```

- This will change the field separator to a comma and print the first two fields from a CSV file.

5. Actions in awk

You can perform actions such as printing, summing, modifying values, etc., in the {} block after the pattern.

Example 1: Sum of Values in a Column

You can calculate the sum of a specific column (e.g., the second column):

```
awk '{ sum += $2 } END { print sum }' file.txt
```

- This sums up the values in the second column and prints the result after processing all lines.

Example 2: Average of a Column

To calculate the average of values in the second column:

```
awk '{ sum += $2 } END { print sum / NR }' file.txt
```

- This calculates and prints the average of the values in the second column.

Example 3: Print Formatted Output

You can format the output using printf (similar to C-style formatting):

```
awk '{ printf "Name: %-10s Age: %-3s\n", $1, $2 }' file.txt
```

- This prints the first and second fields with a fixed-width format, where the name is left-aligned and the age is right-aligned.

6. Using awk for File Processing

Example 1: Find Specific Column in a CSV File

Assume we have a CSV file with fields separated by commas, and we want to extract the third field (column):

```
awk -F, '{ print $3 }' file.csv
```

- This uses the -F option to specify a comma as the field separator.

Example 2: Print Columns Based on a Condition

Print lines where the third column is greater than 50:

```
awk '$3 > 50 { print $1, $2, $3 }' file.csv
```

7. Using awk with Multiple Commands

You can execute multiple actions in an awk script using semicolons ; to separate them.

```
awk '{ print $1; print $2; }' file.txt
```

- This prints the first and second fields of each line, one per line.

Alternatively, you can use the BEGIN and END blocks to define actions before and after processing the input:

Example 1: Use BEGIN and END Blocks

```
awk 'BEGIN { print "Start processing" } { print $1 } END { print "Processing finished" }' file.txt
```

- This prints "Start processing", followed by the first column of each line, and then "Processing finished".

8. Redirection and Piping with awk

You can redirect the output of awk or use it with pipes to work with other commands.

Example 1: Redirect Output to a File

```
awk '{ print $1, $2 }' file.txt > output.txt
```

- This redirects the output of awk to output.txt.

Example 2: Pipe Output to Another Command

You can pipe the output of awk to other commands. For example, to count the unique occurrences of the first field:

```
awk '{ print $1 }' file.txt | sort | uniq -c
```

- This will print the first field of each line, sort the lines, and then count the unique occurrences.

9. awk Script Files

You can store awk commands in a script file instead of using them directly on the command line. The script file typically has a .awk extension.

Example: Create an awk Script

Create a script script.awk:

```
# script.awk
BEGIN { print "Start of file" }
{ print $1, $2 }
END { print "End of file" }
```

Run the script with the following command:

```
awk -f script.awk file.txt
```

10. Conclusion

The awk command is an extremely powerful and flexible tool for text and data processing in Linux. It can be used for a wide range of tasks, from simple data extraction and formatting to complex calculations and text transformations. By learning awk, you gain the ability to automate text processing and analysis tasks efficiently.